AJEENKYA D Y PATIL UNIVERSITY PUNE

SCHOOL OF ENGINEERING

Department of [Department name]

[Insert University Logo here]

A MINI PROJECT REPORT ON

"FAKE REVIEW DETECTOR: AN EXPLAINABLE WEB APPLICATION FOR DETECTING FAKE PRODUCT REVIEWS"

Submitted in partial fulfilment of the requirements of the course

Data Structures using C++ (E0556B)

Second Year B.Tech, Semester [III]

Submitted by

Sr.	Name of student	PRN	Roll No.
1	[Name]	[PRN]	[Roll no.]
2	[Name]	[PRN]	[Roll no.]
3	[Name]	[PRN]	[Roll no.]
4	[Name]	[PRN]	[Roll no.]

Under the guidance of

[Prof. Name of Guide]

Academic Year 20[__] – 20[__]

CERTIFICATE

This is to certify that the mini project entitled "Fake Review Detector: An Explainable Web Application for Detecting Fake Product Reviews" has been carried out and submitted by the following students of Second Year B.Tech, Semester [III], Department of [Department name], School of Engineering, Ajeenkya D Y Patil University Pune, in partial fulfilment of the course Data Structures using C++ (E0556B) during the academic year 20[__] – 20[__]. The work has been found satisfactory.

1.	[Name of student 1]	[PRN]
2.	[Name of student 2]	[PRN]
3.	[Name of student 3]	[PRN]
4.	[Name of student 4]	[PRN]

_____	_____	_____
[Name of Guide] Project Guide	[Name] Head of Department	[Name] Dean, School of Engineering

Place: [_____]

Date: [_____]

DECLARATION

We hereby declare that the mini project report entitled "Fake Review Detector: An Explainable Web Application for Detecting Fake Product Reviews" is our own original work, carried out under the guidance of [Prof. Name of Guide]. The work has not been submitted earlier, in part or in full, to this or any other university or institution for the award of any degree, diploma or credit. All sources of information, code and ideas used have been duly acknowledged in the references.

Sr.	Name of student	PRN	Signature
1	[Name]	[PRN]	
2	[Name]	[PRN]	
3	[Name]	[PRN]	
4	[Name]	[PRN]	

Place: [_____]

Date: [_____]

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, [Prof. Name of Guide], for the constant guidance, encouragement and valuable suggestions given throughout this mini project. The direction we received at every stage, from choosing the problem to refining the detection criteria and preparing this report, helped us turn an idea into a working application.

We are thankful to [Name], Head of the Department of [Department name], for providing the facilities and a supportive environment for project work, and to [Name], Dean, School of Engineering, and the management of Ajeenkya D Y Patil University Pune for giving us the opportunity to work on a practical problem as part of our curriculum.

We also thank the faculty members and the laboratory staff of the department for their help with resources, software and timely feedback during the reviews. Finally, we are grateful to our families and friends for their patience, motivation and support, which kept us going until the project was complete.

[Name 1] • [Name 2] • [Name 3] • [Name 4]

ABSTRACT

Online shoppers depend on product reviews, yet a growing share of reviews are fake: generated by software, written by paid reviewers, or used to advertise. Such reviews mislead buyers and are hard to spot by reading alone. This project presents the Fake Review Detector, a web application in which a user pastes a review or uploads a screenshot of one and receives a FAKE or REAL verdict together with the reasons behind it. The system combines two detectors. A logistic regression classifier over TF-IDF word and character n-gram features, stored as sparse matrices, recognises machine-generated text. Eleven explainable rule checks, built on hash-set word lookups, compiled pattern matching, sentence-level sentiment scoring and a sorted list of matched text intervals, recognise human-written hype, spam, paid-review disclosures and star ratings that contradict the text. Screenshots are converted to text by an on-device optical character recognition engine. The application is written entirely in Python using scikit-learn, VADER, RapidOCR, Plotly and Streamlit, and is deployable free of cost on Streamlit Community Cloud. On 8,087 held-out reviews from a public dataset of 40,432 reviews, the classifier achieved 95.1% accuracy and a ROC AUC of 0.989, while the complete detector caught 96.5% of machine-written fakes at the default setting. The results show that a lightweight, transparent design can detect fake reviews reliably and explain every decision to the user.

Keywords: fake review detection, machine learning, TF-IDF, logistic regression, sentiment analysis, Streamlit

TABLE OF CONTENTS

CERTIFICATE i

DECLARATION ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

CHAPTER 1 INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope and Limitations 2

1.5 Organisation of the Report 3

CHAPTER 2 LITERATURE SURVEY 4

2.1 Existing Systems 4

2.2 Comparative Study 5

2.3 Research Gap 6

CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 7

3.1 Requirement Analysis 7

3.2 Selection of Data Structures 8

3.3 System Architecture and Design 9

3.4 Algorithm and Flowchart 13

3.5 Module Description 14

CHAPTER 4 IMPLEMENTATION 16

4.1 Tools and Technologies 16

4.2 Module-wise Implementation 16

4.3 Sample Input and Output 20

CHAPTER 5 TESTING AND RESULTS 24

5.1 Test Cases 24

5.2 Results and Discussion 25

5.3 Complexity Analysis 28

CHAPTER 6 CONCLUSION AND FUTURE SCOPE 30

6.1 Conclusion 30

6.2 Future Scope 31

REFERENCES 32

APPENDICES 34

Appendix A: Source Code 34

Appendix B: User Manual 60

Appendix C: Plagiarism Report 61

Appendix D: Weekly Progress Log and Team Contribution 62

LIST OF FIGURES

Figure No.	Title of the figure	Page
Figure 3.1	System architecture	10
Figure 3.2	Flowchart of the detection process	14
Figure 3.3	Screenshot-to-text pipeline	13
Figure 4.1	Home screen with input tabs, examples and strictness control	20
Figure 4.2	Verdict card, summary tiles and colour-coded highlights	21
Figure 4.3	Interactive criteria chart	21
Figure 4.4	Sentence-tone chart and session history	22
Figure 4.5	Screenshot analysis with OCR and automatic rating detection	23
Figure 5.1	Detection rates by strictness level	27
Figure 5.2	Confusion matrix of the classifier on 8,087 test reviews	26

LIST OF TABLES

Table No.	Title of the table	Page
Table 2.1	Comparison of existing systems and the proposed system	5
Table 3.1	Hardware and software requirements	8
Table 3.2	Data structures used and justification	8
Table 3.3	Rule-based criteria and weights	11
Table 3.4	Strictness levels and cutoffs	12
Table 3.5	Module description	14
Table 4.1	Tools, libraries and versions	16
Table 5.1	Test cases and results	24
Table 5.2	Performance of the classifier on the held-out test set	25
Table 5.3	Detection rates of the complete detector by strictness	26
Table 5.4	Example cases at the Balanced setting	27
Table 5.5	Time and space complexity of the main operations	28

LIST OF ABBREVIATIONS

Abbreviation	Full form
AUC	Area Under the Curve
CG	Computer-Generated (dataset label for fake reviews)
CSR	Compressed Sparse Row (sparse matrix format)
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
F1	Harmonic mean of precision and recall
HTML	HyperText Markup Language
IDE	Integrated Development Environment
LIFO	Last In, First Out
LRU	Least Recently Used (cache policy)
ML	Machine Learning
NLP	Natural Language Processing
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
OR	Original (dataset label for genuine reviews)
ROC	Receiver Operating Characteristic
SVM	Support Vector Machine
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
VADER	Valence Aware Dictionary and sEntiment Reasoner

CHAPTER 1
INTRODUCTION

1.1 Background

E-commerce has made product reviews one of the most important sources of information for buyers. Before purchasing, most shoppers read the experiences of earlier customers, compare star ratings and look for warnings about quality, delivery or after-sales service. A product with many positive reviews sells more, appears higher in search results and can command a higher price. This economic value has created a strong incentive to manipulate reviews.

Fake reviews appear in several forms. Some are written by people who are paid, or given free products, to praise an item. Some are posted by sellers about their own products, or by competitors to damage a rival. Others are used as advertising and contain links, contact numbers or discount codes. Research on opinion spam began with large-scale studies of Amazon reviews that found duplicate and near-duplicate reviews posted across different products [1]. Later work showed that carefully written deceptive reviews are very difficult for people to identify: in a study of hotel reviews, human judges performed only around chance level, whereas a machine-learning classifier reached close to 90% accuracy [2].

The problem has grown with modern language models, which can produce fluent, believable reviews in large numbers and at almost no cost. Salminen et al. generated 20,216 fake product reviews with a language model, mixed them with an equal number of genuine Amazon reviews and published the collection as an open dataset for detection research [3]. This dataset is used to train the machine-learning part of this project.

The real-world situation modelled by this project is a shopper looking at a single review, either copied as text or captured as a screenshot on a phone, and asking a simple question: can this review be trusted? The Fake Review Detector answers this question in a few seconds and, unlike a black-box score, explains which characteristics of the review led to its verdict.

1.2 Problem Statement

Existing review platforms filter fake reviews using private signals such as reviewer history and purchase records, and the shopper sees only the result, never the reasoning. Independent tools either depend on a particular website or have been discontinued, and machine-learning detectors trained on one kind of fake review often miss other kinds. A shopper therefore has no simple, transparent way to judge an individual review. The proposed system aims to classify any English product review, entered as text or as a screenshot, as FAKE or REAL, and to explain the decision, using a TF-IDF and logistic regression classifier together with explainable rule checks built on hash sets, hash maps, sparse matrices and sorted interval lists.

1.3 Objectives

1. To design a system that accepts a product review as pasted text or as a screenshot image and extracts the review text automatically from images.
2. To implement a machine-learning classifier that detects machine-generated reviews, and to evaluate it on held-out data using accuracy, precision, recall, F1 score and ROC AUC.
3. To implement explainable rule-based checks that detect human-written fake reviews, and to combine them with the classifier so that strong evidence is never diluted.
4. To develop an interactive web interface that shows the verdict, highlights suspicious phrases, charts every criterion and lets the user choose how strict the detector is.
5. To analyse the data structures and algorithms used, test the system systematically and deploy it free of cost on a public cloud platform.

1.4 Scope and Limitations

Scope: The system analyses one English-language product review at a time, such as a review from an e-commerce website. The review can be typed, pasted or uploaded as a PNG, JPG or WEBP screenshot of up to 10 MB. The application runs in any modern web browser on a desktop or mobile phone, requires no installation for the end user and can be used by any shopper, student or researcher.

Limitations: The detector sees only the text of a review. It does not use reviewer history, timing patterns or purchase records, which review platforms use internally. The machine-learning model was trained on computer-generated fakes, so human-written fakes are detected by heuristic rules that indicate suspicion rather than prove it. The system supports English only, and blurred or heavily stylised screenshots may be read incorrectly by the optical character recognition step.

1.5 Organisation of the Report

Chapter 2 reviews existing research and tools for fake review detection and identifies the gap that this project addresses. Chapter 3 analyses the requirements and presents the design: the data structures chosen and why, the system architecture, the main algorithm with its flowchart, and the modules. Chapter 4 describes the implementation module by module with key code listings and screenshots of the working application. Chapter 5 presents the test cases, the experimental results and the complexity analysis. Chapter 6 concludes the report and suggests future improvements. The appendices contain the complete source code, a user manual, the plagiarism report and the weekly progress log.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing Systems

2.1.1 Opinion spam analysis on Amazon

Jindal and Liu carried out one of the first large studies of review spam, using millions of Amazon reviews [1]. They observed that many spam reviews are duplicates or near-duplicates posted by the same or different accounts on different products, and used these duplicates as positive examples to train a supervised logistic regression model. Their features described the review text, the reviewer and the product. The study established review spam as a research problem, but its labels depend on duplication, so original, well-written fake reviews are not captured.

2.1.2 Deceptive opinion spam with linguistic features

Ott et al. built a gold-standard dataset of 400 truthful and 400 deceptive positive hotel reviews, where the deceptive ones were written on request by paid crowd workers [2]. Three human judges could not reliably separate the two classes, performing around chance level. In contrast, a support vector machine trained on word n-grams combined with psycholinguistic features reached nearly 90% accuracy. The study also found that deceptive reviews use more first-person singular pronouns and fewer concrete spatial details, observations that motivate two of the rule checks in this project. The limitation is that the classifier was trained on one domain and on fakes written under controlled conditions.

2.1.3 Behavioural features on real filtered reviews

Mukherjee et al. studied reviews that Yelp's commercial filter had marked as fake [4]. They found that linguistic n-gram features, which worked well on crowd-sourced fakes, performed much worse on these real-world fakes, whereas behavioural features of the reviewer, such as posting bursts, the proportion of extreme ratings and review length patterns, were far more effective. This shows the strength of reviewer behaviour as a signal, but such data is available only to the platform, not to a shopper looking at a single review.

2.1.4 Machine-generated fake reviews

Salminen et al. addressed fake reviews produced by language models [3]. They generated 20,216 reviews with a fine-tuned language model across ten Amazon product categories, combined them with the same number of original reviews, and showed that machine-learning classifiers, in particular a fine-tuned transformer model, can detect generated text with high accuracy. The published dataset is balanced and openly licensed, which makes it suitable for training and evaluating the classifier in this project. Its limitation for a practical tool is that it covers only machine-generated fakes, not human-written paid or spam reviews.

2.1.5 Consumer review-analysis tools

ReviewMeta analyses the reviews of products on Amazon and reports an adjusted rating after discounting reviews that show suspicious patterns, such as unverified purchases, reviewers with unusual histories or bursts of similar reviews [5]. Fakespot offered a browser extension and website that graded the reviews of a product, but it was discontinued by its owner, Mozilla, in 2025 [6]. Both tools work at the level of a whole product on supported websites, require the product page and give limited insight into why an individual review is suspicious.

2.2 Comparative Study

Table 2.1: Comparison of existing systems and the proposed system

Sr.	System / paper	Technique or data structure	Advantages	Limitations
1	Jindal and Liu [1]	Duplicate detection, logistic regression on review, reviewer and product features	Large-scale study; first formal treatment of review spam	Relies on duplicates; misses original fake reviews
2	Ott et al. [2]	Word n-grams and psycholinguistic features with an SVM	Nearly 90% accuracy; far better than human judges	Single domain (hotels); fakes written under controlled conditions
3	Mukherjee et al. [4]	Behavioural features of reviewers	Effective on real filtered fake reviews	Needs reviewer history, which shoppers cannot see
4	Salminen et al. [3]	Generated reviews; transformer and classical classifiers	High accuracy on machine-generated text; open dataset	Covers only machine-generated fakes; heavy models
5	ReviewMeta [5], Fakespot [6]	Product-level analysis of review patterns	Easy for consumers; adjusted product ratings	Site-specific; product-level only; Fakespot discontinued

Sr.	System / paper	Technique or data structure	Advantages	Limitations
6	Proposed system	TF-IDF sparse vectors, logistic regression, 11 rule checks, OCR	Works on any review text or screenshot; explains every verdict; free and lightweight	English only; text-only signals; rules are heuristics

2.3 Research Gap

Existing approaches either depend on data that only platforms hold, such as reviewer behaviour, or specialise in one kind of fake review, and none of the consumer tools explains its decision for an individual review. There is no simple, free tool that accepts a single review from any source, including a screenshot, detects both machine-generated and human-written fakes, and shows the user the evidence. This project addresses the gap by combining a lightweight classifier for machine-generated text with transparent rule checks for human-written fakes in one explainable web application.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Functional requirements:

1. The user shall be able to paste or type review text and request an analysis.
2. The user shall be able to upload a screenshot (PNG, JPG or WEBP, up to 10 MB); the system shall extract the review text and allow the user to correct it before analysis.
3. The system shall detect the star rating when the screenshot shows it in the form "x out of 5", and let the user enter a rating for pasted text.
4. The system shall reject input with fewer than three words and show a clear message.
5. The system shall display a FAKE or REAL verdict, a fake score from 0% to 100% and a confidence level.
6. The system shall explain the verdict: the score and reason for each criterion, and the phrases that triggered a rule highlighted in the review.
7. The user shall be able to choose a strictness level (Relaxed, Balanced or Strict), and the verdict shall update immediately.
8. The system shall keep a history of the reviews checked during the session.

Non-functional requirements:

- **Performance:** a pasted review shall be analysed in well under a second; a screenshot within a few seconds on the free cloud tier.
- **Usability:** the interface shall be understandable without training, work on desktop and mobile screens, and support light and dark themes.
- **Explainability:** every verdict shall be accompanied by plain-English reasons.
- **Reliability:** the behaviour shall be covered by automated tests, and the model shall load from a version-pinned file.

- **Portability:** the system shall run on Windows and Linux with Python 3.11 and need no system packages or external services.
- **Privacy and security:** reviews shall be processed in memory and never stored or sent to third parties; review text shall be escaped before display so that it cannot inject content into the page.

Table 3.1: Hardware and software requirements

Category	Specification
Processor and RAM	Any 64-bit dual-core processor; 4 GB RAM minimum, 8 GB recommended
Operating system	Windows 10/11 (development); Linux container on Streamlit Community Cloud (deployment)
Compiler / IDE	Python 3.11 interpreter; Visual Studio Code
Language	Python 3.11 (application); HTML and CSS (interface styling)
Libraries	Streamlit 1.66, scikit-learn 1.9, VADER 3.3, RapidOCR 3.10 with ONNX Runtime 1.31, Plotly 7.1, pandas 3.1, NumPy 2.4, Pillow 13.0
Other tools	Git and GitHub (version control), pytest (testing), Microsoft Word (report)
Client	Any modern web browser on a desktop or mobile phone

3.2 Selection of Data Structures

The performance and clarity of the detector depend on choosing the right data structure for each task. Most checks must look up every word of a review in a word list, so constant-time lookup is essential. The machine-learning features have more than one hundred thousand dimensions but only a few hundred non-zero values per review, so a sparse representation is needed. Table 3.2 lists the data structures used, where they are used and why they were chosen [10], [11].

Table 3.2: Data structures used and justification

Data structure	Where used in the project	Why it was chosen	Time complexity of key operations
Hash set (Python set)	Word lists: extreme words, first-person pronouns, product aspects, OCR noise words	Membership must be tested for every word of the review	Lookup O(1) average

Data structure	Where used in the project	Why it was chosen	Time complexity of key operations
Hash map (Python dict)	VADER sentiment lexicon, TF-IDF vocabulary (term to column), strictness levels, session state	Fast key-to-value lookup for thousands of entries	Lookup and insert O(1) average
Sparse matrix (CSR)	TF-IDF feature vector of each review (120,000 columns)	Stores only non-zero values, saving memory and time	Dot product O(nnz)
Dynamic array (Python list)	Tokens, sentences, list of signal results, OCR text lines	Ordered data that is appended and iterated	Append O(1) amortised; scan O(n)
Sorted list of intervals	Character spans of phrases to highlight	Enables a single left-to-right sweep that skips overlaps	Sort O(m log m); sweep O(m)
Set of tuples	Word pairs (bigrams) in the repetition check	Counts repeated pairs by comparing set size with list size	Build O(n)
Compiled regular expression (finite automaton)	Stock phrases, links and promo codes, incentive disclosures, star rating in OCR text	Matches many alternative phrases in one pass over the text	About O(n) per pattern for these patterns
LRU cache (hash map with doubly linked list)	Loading the model and the OCR engine once	Expensive objects are built once and reused	Cache hit O(1)
Record (dataclass)	Signal, Analysis and OcrResult objects	Groups related fields into one value	Field access O(1)
List used as a stack (newest first)	Session history of checked reviews	Most recent check is shown first (LIFO order)	Insert at front O(h)

Two design decisions deserve explanation. First, the hash set is preferred over a sorted array with binary search because there are no range queries; each word only needs an exact membership test, which a hash set answers in constant average time, compared with O(log n) for binary search. Second, the session history is a list in which each new check is inserted at the front. This costs O(h) for h stored items, but the history of a single session is small, so the simpler structure was preferred; a double-ended queue (collections.deque) would make the insertion O(1) if the history were large.

3.3 System Architecture and Design

The system follows a layered architecture, shown in Figure 3.1. The presentation layer is the Streamlit page together with a module of rendering components. The processing

layer contains the analyser, which validates the input and coordinates the two detectors, and the OCR module, which converts screenshots to text. The model layer contains the trained classifier and the rule checks. Data flows from the user through the interface to the analyser, and the verdict with its explanations flows back to the interface.

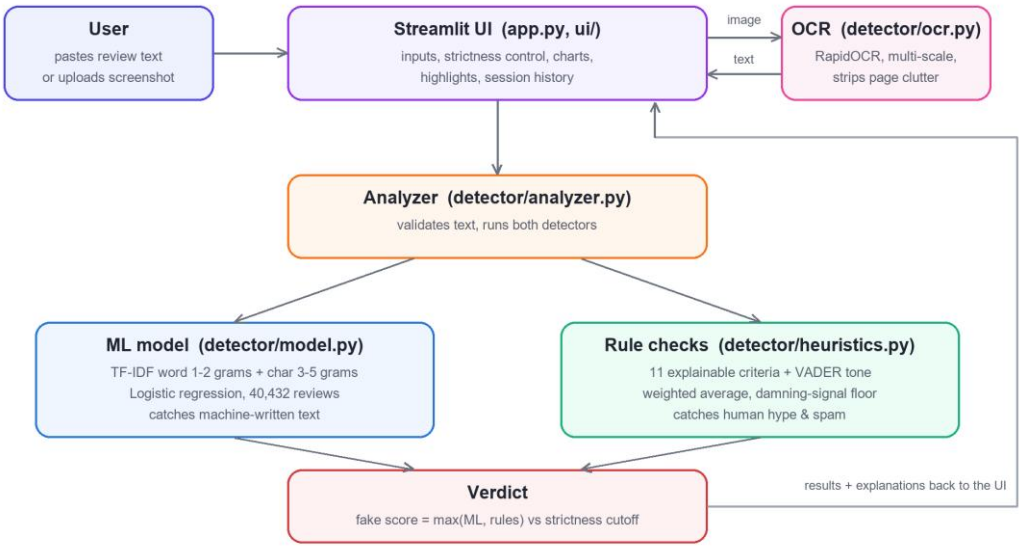


Figure 3.1: System architecture

When the user uploads an image, the interface first sends it to the OCR module, which returns the review text and any star rating found. The text, whether typed or extracted, is passed to the analyser. The analyser runs the machine-learning model, which estimates the probability that the text was machine-generated, and the rule checks, which produce eleven scored criteria. The two results are combined into a single fake score, compared with the cutoff for the chosen strictness, and returned to the interface for display. The detectors have no knowledge of the interface, so they can be tested independently and reused, for example in a batch tool.

3.3.1 Machine-learning model

Each review is converted into a numeric vector using term frequency–inverse document frequency (TF-IDF) weighting [7]. For a term t in review d, the weight is tf(t, d) × idf(t), where tf is the (log-scaled) number of times t occurs in d and idf(t) = log((1 + N) / (1 + df(t))) + 1, with N the number of training reviews and df(t) the number of reviews containing t. Rare, distinctive terms therefore receive higher weights than common words. Two vocabularies are used: word unigrams and bigrams, which capture phrasing, and character sequences of three to five characters, which capture spelling,

punctuation and stylistic patterns typical of generated text. Each vocabulary is limited to 60,000 terms.

A logistic regression classifier [8] computes the probability that a review is machine-generated as p = 1 / (1 + e^-(w·x + b)), where x is the sparse TF-IDF vector, w the learned weights and b the bias. Logistic regression was chosen because it trains quickly on sparse, high-dimensional text features, its prediction is a single sparse dot product, and it outputs a probability that can be compared with a threshold.

3.3.2 Rule-based criteria

The rule checks capture traits of human-written fake reviews reported in the literature and observed in practice. Each check returns a score between 0 (looks genuine) and 1 (looks fake), a weight, and a plain-English reason. Three checks are evidence-only: they count only when they fire, because the absence of a spam link, for example, says nothing about whether a review is genuine. Table 3.3 lists the criteria. Sentiment is measured with VADER, a lexicon- and rule-based sentiment analyser [9], applied sentence by sentence so that a negation in one sentence cannot change the meaning of the next.

Table 3.3: Rule-based criteria and weights

Criterion	What it detects	Weight	Type
Extreme / superlative words	Density of words such as amazing, perfect, worst	1.2	Scored
Stock / template phrases	Phrases such as “highly recommend”, “worth every penny”	1.2	Scored
Excessive punctuation / CAPS	Exclamation marks per sentence and all-capital words	0.8	Scored
Self-focused language	Share of I, me, my (more frequent in deceptive reviews [2])	0.5	Scored
Lack of concrete details	Few product features, numbers or durations of use	1.2	Scored
One-sided sentiment	Entirely positive or negative with no trade-offs	1.0	Scored
Very short review	Too few words to show real use	0.7	Scored
Repetitive wording	Low vocabulary variety and repeated word pairs	0.8	Scored
Links / promo codes / contact info	URLs, e-mail addresses, phone numbers, discount codes	1.5	Evidence-only
Incentivised review disclosure	“Received free in exchange for an honest review”	1.0	Evidence-only

Criterion	What it detects	Weight	Type
Star rating vs. text mismatch	A 5-star complaint or a 1-star rave	1.5	Evidence-only

3.3.3 Combining the scores

The rule score is the weighted average of the criteria that carry evidence. A safeguard ensures that strong evidence is never diluted: the rule score is at least as high as the strongest evidence-only signal. Without this safeguard, a spam review containing a discount code and a shop link scored only 53%, because the many checks that found nothing pulled the average down. The final fake score is the maximum of the model probability and the rule score, because the two detectors look for different kinds of fake; averaging them would hide fakes that only one detector can recognise. The strictness setting determines the cutoff, as shown in Table 3.4.

Table 3.4: Strictness levels and cutoffs

Strictness	Flag as fake when the score is	Recommended use
Relaxed	65% or more	Only clear-cut cases should be flagged
Balanced (default)	50% or more	General use
Strict	35% or more	Catching more fakes matters more than false alarms

Confidence is the distance between the score and the cutoff, scaled to the range 50% to 100%. For a FAKE verdict it is $0.5 + 0.5 \times (p - t) / (1 - t)$, and for a REAL verdict $0.5 + 0.5 \times (t - p) / t$, where p is the fake score and t the cutoff. A verdict is therefore never reported with less than 50% confidence, whichever strictness is selected.

3.3.4 Screenshot processing

Screenshots are processed by the pipeline in Figure 3.3. The OCR engine occasionally misreads a line at one image size and reads it perfectly at another, and in such cases it reports a low confidence score. The image is therefore read at up to three scales, and the reading with the highest confidence, weighted by text length, is kept. Text fragments on the same visual row are merged, rows are ordered from top to bottom, and website

clutter such as “Helpful”, “Report” and “12 people found this helpful” is removed. The star rating and the “Verified Purchase” badge are extracted separately.

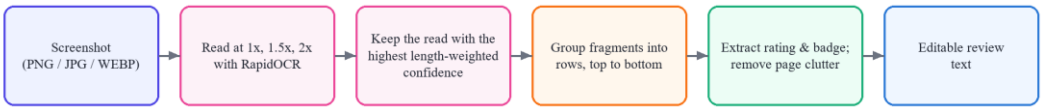


Figure 3.3: Screenshot-to-text pipeline

3.4 Algorithm and Flowchart

The main detection algorithm is given below in numbered steps, and its flowchart is shown in Figure 3.2.

Algorithm: DetectFakeReview(input, rating, strictness)

1. Start.
2. If the input is an image, run the OCR pipeline (Figure 3.3) to obtain the review text and the star rating, and let the user correct the text.
3. Split the text into words. If there are fewer than three words, display an error message and stop.
4. Compute the TF-IDF sparse vector x of the text and the model probability p_ml = sigmoid(w·x + b).
5. For each of the eleven criteria, compute a score s_i in [0, 1] with weight w_i and record the matched text spans.
6. Compute the rule score p_rule = max(Σ w_i s_i / Σ w_i over criteria with evidence , highest evidence-only score).
7. Compute the fake score p = max(p_ml, p_rule).
8. Look up the cutoff t for the chosen strictness (0.65, 0.50 or 0.35).
9. If p ≥ t, the verdict is FAKE; otherwise it is REAL. Compute the confidence from p and t.
10. Sort the matched spans, highlight non-overlapping phrases, draw the charts and add the result to the session history.
11. Stop.

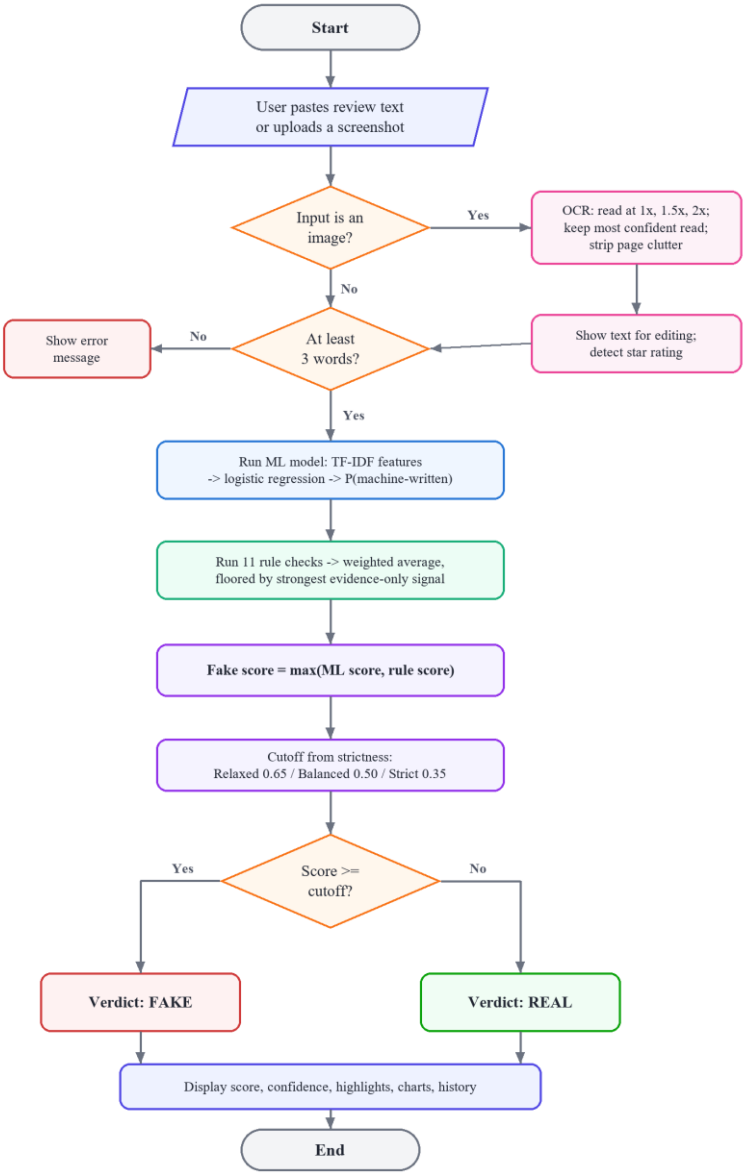


Figure 3.2: Flowchart of the detection process

3.5 Module Description

The application is divided into the modules listed in Table 3.5. Each module has a single responsibility, which keeps the code easy to test and allows team members to work on different modules in parallel.

Table 3.5: Module description

Module	Purpose	Data structure used	Team member
app.py (interface)	Page layout, input tabs, strictness control, session history	Hash map (session state), list as stack	[Name]
ui/components.py	Verdict card, tiles, highlighted text, interactive charts	Sorted interval list, lists	[Name]

Module	Purpose	Data structure used	Team member
detector/analyzer.py	Validates input, runs both detectors, computes verdict and confidence	Record (dataclass)	[Name]
detector/heuristics.py	Eleven rule checks and the score combination	Hash sets, hash map, regex automata, set of tuples	[Name]
detector/model.py	Loads the classifier and returns the model probability	Sparse matrix, LRU cache	[Name]
detector/ocr.py	Screenshot to review text, rating and badge	Lists, regex, LRU cache	[Name]
train_model.py	Trains, evaluates and saves the model	Sparse matrix, hash map (vocabulary)	[Name]
tests/	Automated unit and end-to-end tests	Lists	[Name]

CHAPTER 4
IMPLEMENTATION

4.1 Tools and Technologies

All tools and libraries used are free and open source. Python was chosen because it offers mature libraries for machine learning, natural language processing and web interfaces, so the whole project can be written in one language. Table 4.1 lists the versions used, which are pinned in the requirements file so that the deployed application behaves exactly like the tested one.

Table 4.1: Tools, libraries and versions

Tool / library	Version	Role in the project
Python	3.11	Programming language for all modules
Streamlit [13]	1.66.1	Turns the Python script into an interactive web page
scikit-learn [12]	1.9.1	TF-IDF vectorisers, logistic regression, evaluation metrics
VADER [9]	3.3.2	Sentence-level sentiment scores
RapidOCR [14]	3.10.0	Text detection and recognition in screenshots (PP-OCR models [15])
ONNX Runtime	1.31.0	Runs the OCR neural networks on the CPU
Plotly [16]	7.1.0	Interactive bar charts with hover details
pandas / NumPy	3.1.0 / 2.4.6	Dataset loading, arrays, the history table
Pillow	13.0.0	Image loading and resizing
pytest	latest	Unit and end-to-end tests
Git and GitHub	–	Version control and hosting of the source code
Visual Studio Code	–	Code editor

4.2 Module-wise Implementation

4.2.1 Model training (train_model.py)

The training script loads the dataset of 40,432 reviews, maps the label CG (computer-generated) to 1 and OR (original) to 0, and splits the data into 80% training and 20% test sets with the same class proportions (stratified split). The feature pipeline in Listing 4.1 combines the word and character TF-IDF vectorisers; a logistic regression classifier

with regularisation parameter C = 4 is trained on top. After evaluation on the test set, the model is refitted on all data and saved as a compressed file of 2.2 MB, together with its metrics.

```
def build_pipeline() -> Pipeline:
    features = FeatureUnion([
        ("words", TfidfVectorizer(
            ngram_range=(1, 2), min_df=3, max_features=60_000,
            sublinear_tf=True, strip_accents="unicode")),
        ("chars", TfidfVectorizer(
            analyzer="char_wb", ngram_range=(3, 5), min_df=5,
            max_features=60_000, sublinear_tf=True)),
    ])
    return Pipeline([
        ("features", features),
        ("clf", LogisticRegression(C=4.0, max_iter=2000)),
    ])
```

Listing 4.1: TF-IDF feature pipeline (train_model.py)

4.2.2 Rule checks (detector/heuristics.py)

Each criterion is a function that receives the review text and its list of word matches, and returns a Signal record holding the score, weight, explanation and the character spans to highlight. Word lists are stored as hash sets, so each membership test takes constant time. Listing 4.2 shows the extreme-word check: it counts the words that appear in the set of extreme words and converts their density into a score, so that a review in which 8% or more of the words are extreme scores 1.

```
HYPE_WORDS = {"amazing", "perfect", "worst", ...}       # hash set

def hype_words(text: str, words: list[re.Match]) -> Signal:
    # each "in HYPE_WORDS" membership test is O(1) on average
    hits = [m for m in words if m.group().lower() in HYPE_WORDS]
    density = len(hits) / max(1, len(words))
    score = _clamp(density / 0.08)
    return Signal(
        "hype", "Extreme / superlative words", score, 1.2,
        f"{len(hits)} extreme word(s) ({density:.0%} of the text).",
        spans=[m.span() for m in hits],
    )
```

Listing 4.2: Extreme-word check using a hash set (heuristics.py)

The scores are combined by the function in Listing 4.3, which implements the weighted average with the evidence-only safeguard described in Section 3.3.3.

```
def combine(signals: list[Signal]) -> float:
    """Weighted average of the signals that carry evidence,
    floored by the strongest evidence-only signal."""
    active = [s for s in signals if s.counts]
    total = sum(s.weight for s in active)
    average = (sum(s.score * s.weight for s in active) / total
               if total else 0.0)
    strongest = max((s.score for s in signals if s.evidence_only),
                    default=0.0)
    return max(average, strongest)
```

Listing 4.3: Combining the rule scores (heuristics.py)

4.2.3 Analyser (detector/analyzer.py)

The analyser validates the input, runs both detectors and returns an Analysis record. Listing 4.4 shows how the final score is the maximum of the two detector scores. The Analysis record also computes the verdict and the confidence from the threshold, so changing the strictness only requires replacing the threshold, not repeating the analysis.

```
def analyze(text: str, rating: int | None = None) -> Analysis:
    text = text.strip()
    if len(heuristics.WORD_RE.findall(text)) < MIN_WORDS:
        raise ValueError(f"Please provide at least {MIN_WORDS} "
                         "words of review text.")

    signals = heuristics.run_signals(text, rating)
    rule_score = heuristics.combine(signals)
    ml_prob = model.fake_probability(text)
    # The model spots machine-written text and the rules spot
    # human-written hype or spam: either being confident is enough.
    final = (rule_score if ml_prob is None
             else max(ml_prob, rule_score))
    return Analysis(text, final, ml_prob, rule_score, signals)
```

Listing 4.4: Running and combining the two detectors (analyzer.py)

4.2.4 OCR (detector/ocr.py)

Listing 4.5 shows the multi-scale reading. The image is read at scales 1, 1.5 and 2, skipping scales that would make the image larger than 4,000 pixels. For each reading, the confidence scores of the text lines are averaged, weighted by line length, and the best reading is kept. The loop stops early as soon as a reading is confident enough, which saves time on clear screenshots.

```
def _best_read(image: Image.Image):
    """OCR at a few scales; return the most confident output."""
    image = image.convert("RGB")
    best, best_score = None, -1.0
    for scale in SCALES:                          # 1.0, 1.5, 2.0
        if scale > 1 and max(image.size) * scale > MAX_SIDE:
            break
        size = (round(image.width * scale),
                round(image.height * scale))
        scaled = (image if scale == 1
                  else image.resize(size, Image.LANCZOS))
        out = _engine()(np.array(scaled))
        if out.boxes is None:
            continue
        chars = max(1, sum(map(len, out.txts)))
        score = sum(s * len(t)
                    for s, t in zip(out.scores, out.txts)) / chars
        if score > best_score:
            best, best_score = out, score
        if score >= GOOD_ENOUGH and min(out.scores) >= 0.8:
            break                          # confident: stop early
    return best
```

Listing 4.5: Multi-scale OCR reading (ocr.py)

4.2.5 Highlighting (ui/components.py)

To highlight suspicious phrases, the character spans from all triggered criteria are collected into a list of intervals and sorted by start position. A single left-to-right sweep then builds the output: text between phrases is escaped and copied, each phrase is wrapped in a coloured mark, and any interval that starts before the end of the previous one is skipped. Escaping every piece of user text prevents a malicious review from injecting HTML into the page.

```
spans = sorted(                                    # O(m log m)
    (start, end, s.key, s.name)
    for s in a.signals if s.triggered and s.key in HIGHLIGHTS
    for start, end in s.spans)
out, pos = [], 0
for start, end, key, name in spans:                # O(m) sweep
    if start < pos:
        continue                  # overlaps a phrase already marked
    out.append(html.escape(a.text[pos:start]))
    phrase = html.escape(a.text[start:end])
    out.append(f'<mark class="m-{key}">{phrase}</mark>')
    pos = end
out.append(html.escape(a.text[pos:]))
```

Listing 4.6: Sorted-interval sweep for highlighting (components.py)

4.2.6 User interface (app.py and ui/)

The interface is a Streamlit script. Streamlit re-runs the script from top to bottom whenever the user interacts with a widget, so results and history are kept in the session state, a hash map that survives re-runs. The page has two input tabs (paste review and

upload screenshot), example buttons that fill the text box and analyse it immediately, and a sidebar with the strictness control and model performance. Results are shown as an animated verdict card, four summary tiles and four views: highlights, an interactive criteria chart, a sentence-tone chart and a detailed list. Custom CSS gives the page a gradient header, rounded cards and colours that adapt to light and dark themes.

4.3 Sample Input and Output

Figure 4.1 shows the home screen. The user can paste a review or click one of the examples; the sidebar contains the strictness control and the model's accuracy on held-out data.

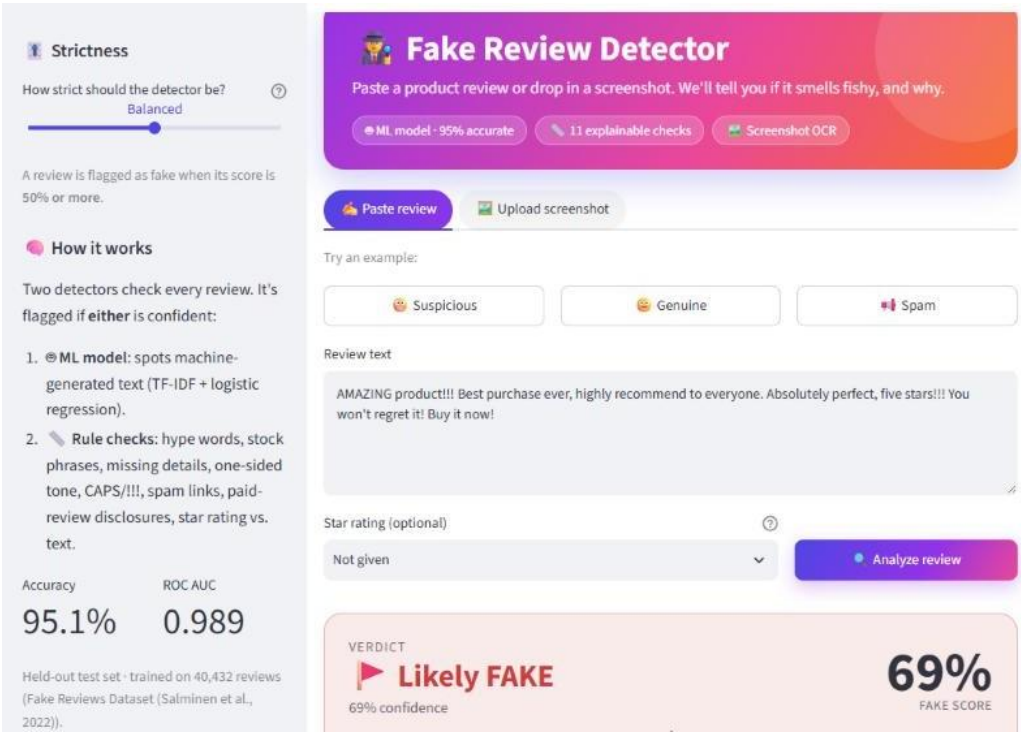


Figure 4.1: Home screen with input tabs, examples and strictness control

For the input "AMAZING product!!! Best purchase ever, highly recommend to everyone. Absolutely perfect, five stars!!! You won't regret it! Buy it now!" the system reports a fake score of 69% and the verdict FAKE (Figure 4.2). The model score is 0%, because the review was written by a person, but the rule checks score 69% because of extreme words, stock phrases, excessive punctuation and a lack of concrete details. The highlights view marks each triggering phrase in the colour of its criterion.

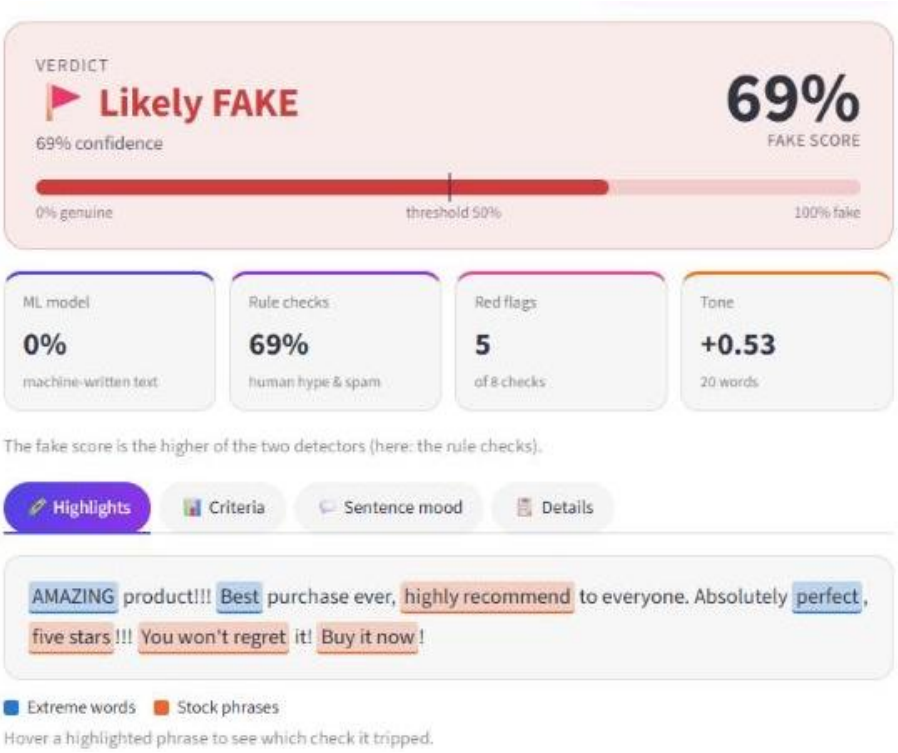


Figure 4.2: Verdict card, summary tiles and colour-coded highlights

The criteria view (Figure 4.3) charts every check. Bars are coloured by status and labelled with matching icons, and hovering over a bar shows the reasoning behind its score.

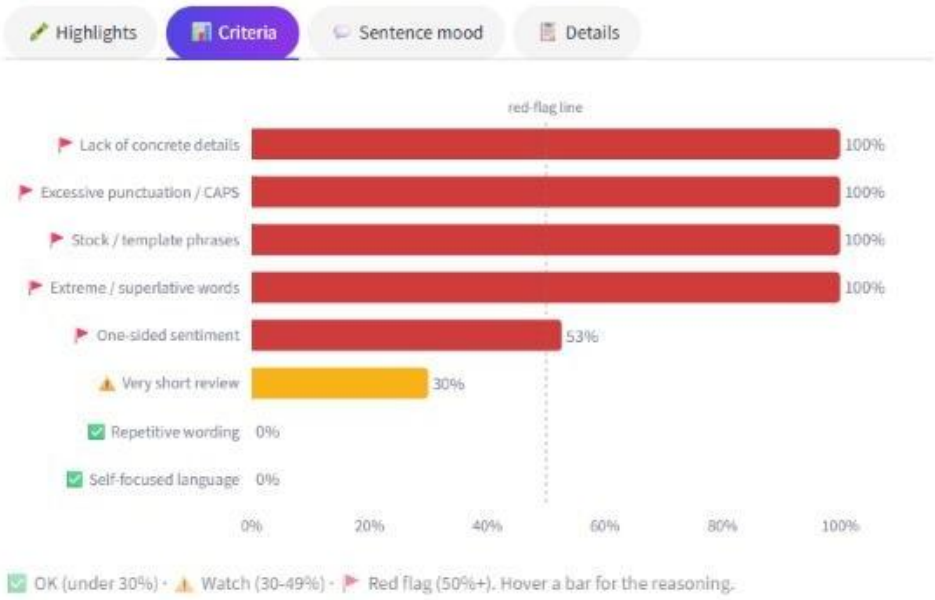


Figure 4.3: Interactive criteria chart

The sentence-mood view (Figure 4.4) charts the tone of each sentence from negative to positive, shown here for a genuine review about a kettle. Below it, the session history lists every review checked.

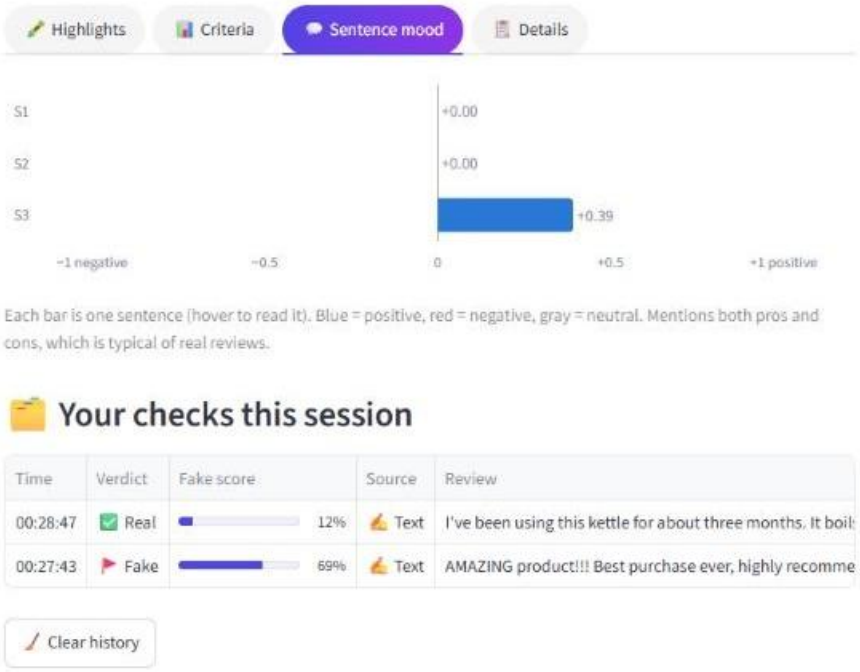


Figure 4.4: Sentence-tone chart and session history

Figure 4.5 shows the screenshot tab. The OCR module has read the review text, removed the website clutter and detected the 5-star rating. Because the review contains a link to a shop, the detector reports FAKE with a score of 75%.

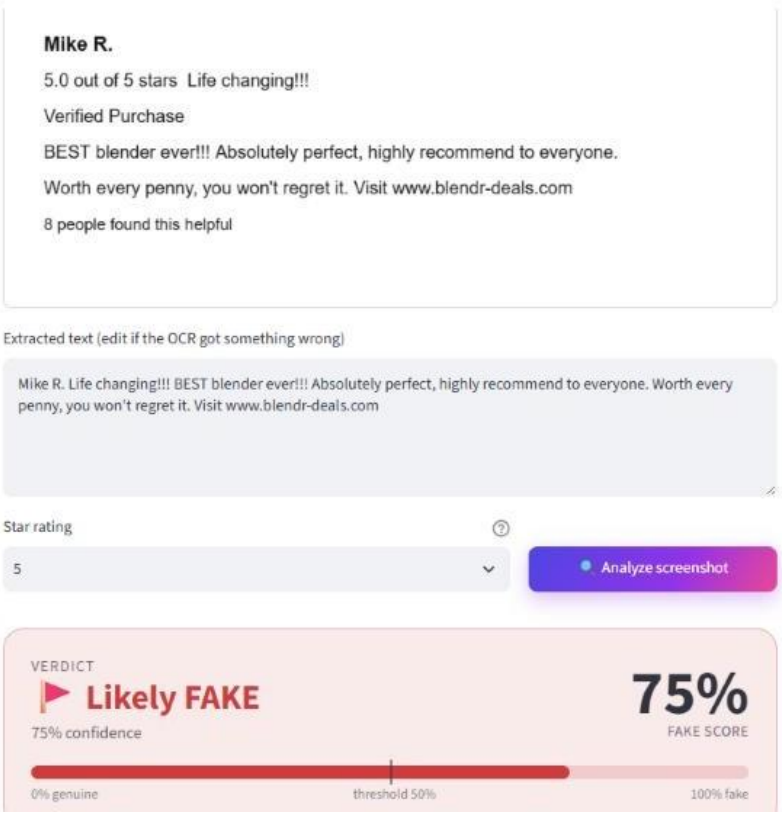


Figure 4.5: Screenshot analysis with OCR and automatic rating detection

CHAPTER 5
TESTING AND RESULTS

5.1 Test Cases

The system was tested at three levels. Unit tests check individual criteria and the analyser, an OCR test checks the screenshot pipeline on a sample image, and an end-to-end test runs the complete Streamlit page headless using Streamlit's testing framework, clicks an example and checks the result. In total, 15 automated tests were run with pytest after a clean installation of the pinned requirements, and all passed. Interface behaviour was additionally tested manually in a web browser. Table 5.1 lists the test cases.

Table 5.1: Test cases and results

Test ID	Description	Input	Expected output	Actual output	Pass / Fail
T1	Hype review is flagged	"AMAZING product!!! Best purchase ever...", 5 stars	FAKE	FAKE, score 69%	Pass
T2	Spam review is flagged	Review with code SAVE20 and a shop URL	FAKE	FAKE, score 95%	Pass
T3	Spam is not averaged away when Relaxed	Same spam review, cutoff 0.65	FAKE, score ≥ 90%	FAKE, score 95%	Pass
T4	Detailed, balanced review is accepted	Kettle review with numbers, pros and cons	REAL	REAL, score 12%	Pass
T5	Rating mismatch is detected	"Broke after two days... Terrible.", 5 stars	Mismatch criterion fires	Fires; FAKE, score 90%	Pass
T6	Too-short input is rejected	"Nice"	Error message	"Please provide at least 3 words"	Pass
T7	ISBN is not mistaken for a phone number	Book review containing ISBN 0307887898	Promo criterion does not fire	Does not fire	Pass
T8	Confidence never below 50%	Two reviews × three cutoffs (6 cases)	Confidence ≥ 50%	All 6 ≥ 50%	Pass
T9	Screenshot is read correctly	Sample review screenshot	Text read, clutter removed, rating 5, badge found	As expected	Pass
T10	Image without text	Blank white image	"No readable text" warning	Warning shown	Pass
T11	Full page works end to end	Click the Suspicious example	FAKE verdict and one history row	As expected	Pass
T12	Incentivised review is flagged	"I received this product for free in exchange..."	FAKE	FAKE, score 87%	Pass

Test ID	Description	Input	Expected output	Actual output	Pass / Fail
T13	Strictness updates verdict instantly	"Great product, works great, love it..." (score 43%); Balanced then Strict	REAL, then FAKE, without re-analysis	REAL, then FAKE	Pass
T14	HTML in a review is not executed	Review containing a <script> tag	Shown as plain text	Shown as plain text	Pass
T15	Responsive and dark themes	Phone width (375 px); dark mode	Readable; no horizontal scroll	As expected	Pass

5.2 Results and Discussion

5.2.1 Performance of the machine-learning model

The model was evaluated on the 8,087 test reviews that were held out from training. Table 5.2 summarises the metrics. Precision is the share of reviews predicted as fake that really are fake; recall is the share of fake reviews that were found; the F1 score is their harmonic mean; and the ROC AUC measures how well the predicted probabilities rank fake reviews above real ones across all thresholds, where 1.0 is perfect and 0.5 is random.

Table 5.2: Performance of the classifier on the held-out test set

Metric	Real reviews	Fake reviews	Overall
Precision	94.4%	95.8%	–
Recall	95.8%	94.3%	–
F1 score	0.951	0.950	0.950
Accuracy	–	–	95.1%
ROC AUC	–	–	0.989

The confusion matrix in Figure 5.2 shows that, out of 4,044 genuine test reviews, 3,876 were classified correctly and 168 were false alarms, and out of 4,043 machine-generated reviews, 3,813 were caught and 230 were missed. The errors are almost balanced between the two classes, which shows that the model is not biased towards either label.

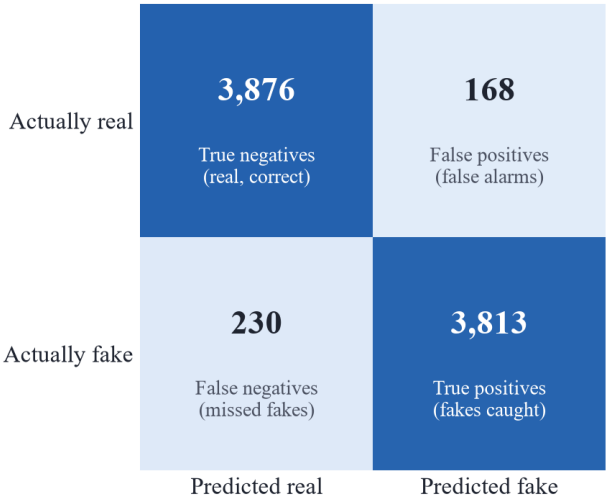


Figure 5.2: Confusion matrix of the classifier on 8,087 test reviews

5.2.2 Effect of the strictness setting

The complete detector, combining the model and the rule checks, was run on a random sample of about 3,000 reviews from the dataset at each strictness level. Table 5.3 and Figure 5.1 show the trade-off: a stricter setting catches more machine-written fakes but also flags more genuine reviews.

Table 5.3: Detection rates of the complete detector by strictness

Strictness	Genuine reviews flagged as fake	Machine-written fakes caught
Relaxed (0.65)	2.8%	93.4%
Balanced (0.50)	4.7%	96.5%
Strict (0.35)	12.2%	98.4%

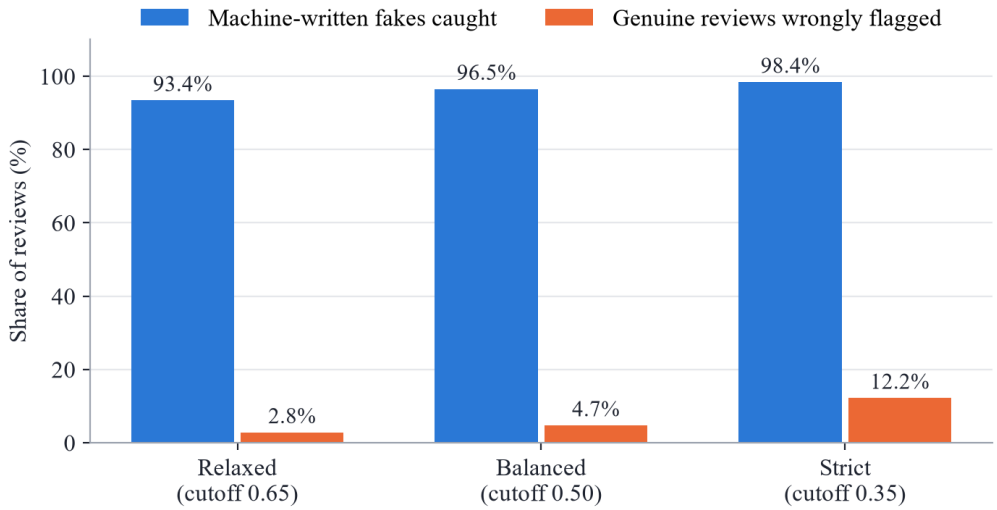


Figure 5.1: Detection rates by strictness level

At the Balanced setting, the detector flags 4.7% of the dataset's genuine reviews. Inspection showed that many of these openly disclose being incentivised, for example "received this item at a discount in exchange for an honest review". They were written by people, so the dataset labels them genuine, but they are not independent opinions, and flagging them is the intended behaviour. One real false alarm, a book's ISBN number matched as a phone number, led to a stricter phone-number pattern, now covered by test T7.

5.2.3 Example cases

Table 5.4 shows how the two detectors complement each other. The human-written hype review receives 0% from the model but 69% from the rules; the incentivised review is caught mainly by the model. The genuine reviews receive low scores from both detectors, even when one of them is a strong complaint with a 1-star rating, because the rating matches the tone of the text.

Table 5.4: Example cases at the Balanced setting

Review	Model	Rules	Final	Verdict	Main reasons
Hype review	0%	69%	69%	FAKE	Extreme words, stock phrases
Spam with promo code	29%	95%	95%	FAKE	Link and promo code, no concrete details
Incentivised review	87%	65%	87%	FAKE	Paid-review disclosure, one-sided tone
5-star complaint	1%	90%	90%	FAKE	Rating contradicts text, extreme words

Review	Model	Rules	Final	Verdict	Main reasons
Detailed genuine review	12%	1%	12%	REAL	Specific details, pros and cons
Genuine 1-star complaint (screenshot)	2%	7%	7%	REAL	Specific details, rating matches text

Observed behaviour matched the expected behaviour in every test case. The main weakness found during testing was the dilution of strong evidence in the weighted average, which was corrected by the evidence-only safeguard. The remaining limitation is that the rule checks are heuristics: a skilled writer can produce a fake review with specific details, balanced sentiment and no promotional content, and such a review may not be detected from its text alone.

5.3 Complexity Analysis

Let n be the number of characters in a review, k the number of criteria (11), m the number of highlighted spans, V the vocabulary size (120,000 features), z the number of non-zero features of a review, h the number of items in the session history and P the number of pixels in a screenshot. Table 5.5 lists the cost of each operation.

Table 5.5: Time and space complexity of the main operations

Operation	Time complexity	Space complexity
Tokenising the review into words and sentences	O(n)	O(n)
Word-list checks (hash-set lookups)	O(n)	O(1) extra
Pattern checks (compiled regular expressions)	About O(n) per pattern	O(1) extra
Repetition check (set of word pairs)	O(n)	O(n)
Sentiment scoring (VADER, per sentence)	O(n)	O(n)
Combining k criteria	O(k)	O(k)
TF-IDF transformation (word and character n-grams)	O(n)	O(z)
Logistic regression prediction (sparse dot product)	O(z)	O(V) for the stored weights
Highlighting (sort and sweep of spans)	O(m log m)	O(m + n)
Adding to session history (insert at front)	O(h)	O(h)

Operation	Time complexity	Space complexity
OCR of a screenshot (up to three scales)	O(P) per scale	O(P)

Since k is a constant and $z \leq n$, the analysis of a text review runs in linear time, O(n), apart from the small sorting step for highlights. In practice, a pasted review is analysed in a fraction of a second. Reading a screenshot is the most expensive operation because it runs two neural networks over the image; the early-exit condition in Listing 4.5 means most clear screenshots are read only once.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This mini project developed the Fake Review Detector, an explainable web application that classifies product reviews as FAKE or REAL from pasted text or screenshots. Each objective set in Section 1.3 was achieved.

Objective 1, input handling: the system accepts typed or pasted text and screenshots in common image formats. A multi-scale OCR pipeline extracts the review text, removes website clutter and detects the star rating and the Verified Purchase badge, and the user can correct the extracted text before analysis.

Objective 2, machine-learning classifier: a TF-IDF and logistic regression model trained on 32,345 reviews achieved 95.1% accuracy, an F1 score of 0.950 and a ROC AUC of 0.989 on 8,087 held-out reviews. The model is only 2.2 MB and predicts with a single sparse dot product.

Objective 3, explainable rules: eleven rule checks detect human-written fakes that the model cannot see, such as hype, template phrases, spam links, paid-review disclosures and rating mismatches. Combining the detectors by taking the maximum, and flooring the rule score by the strongest evidence-only signal, ensures that strong evidence is never diluted.

Objective 4, interactive interface: the Streamlit interface shows an animated verdict with a confidence level, highlights the triggering phrases, charts every criterion and the tone of each sentence, keeps a session history, and lets the user switch between Relaxed, Balanced and Strict with an immediate update. It works on phones and in light and dark themes.

Objective 5, analysis, testing and deployment: the data structures were chosen for constant-time lookups, sparse storage and linear-time processing, giving an overall O(n) analysis of a text review. Fifteen automated tests pass after a clean installation, and the application is deployable free of cost on Streamlit Community Cloud [17], with its source code hosted on GitHub.

The project shows that a lightweight design built on well-chosen data structures and transparent rules can detect fake reviews reliably, and that explaining a decision is as valuable to a user as the decision itself.

6.2 Future Scope

- **Training on human-written fakes:** add datasets of deceptive reviews written by people, such as the hotel corpus of Ott et al. [2], so that the model also learns human-written fakes and depends less on heuristics.
- **Transformer models:** compare a fine-tuned transformer such as DistilBERT with the current model in terms of accuracy, speed and memory on the free hosting tier.
- **Batch analysis:** allow a CSV file of reviews to be uploaded and show a summary dashboard, using a priority queue to list the most suspicious reviews first.
- **Campaign detection:** analyse all the reviews of a product together, using hashing (for example MinHash) to find near-duplicate reviews posted as part of coordinated campaigns.
- **More languages and a browser extension:** support Hindi and other languages, and provide a browser extension that checks reviews directly on shopping websites.

REFERENCES

[1] N. Jindal and B. Liu, "Opinion spam and analysis," in Proc. Int. Conf. Web Search and Data Mining (WSDM), Palo Alto, CA, USA, 2008, pp. 219–230.

[2] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, "Finding deceptive opinion spam by any stretch of the imagination," in Proc. 49th Annu. Meeting Assoc. Comput. Linguistics (ACL), Portland, OR, USA, 2011, pp. 309–319.

[3] J. Salminen, C. Kandpal, A. M. Kamel, S. Jung, and B. J. Jansen, "Creating and detecting fake reviews of online products," Journal of Retailing and Consumer Services, vol. 64, Art. no. 102771, Jan. 2022. Dataset: https://osf.io/tyue9/

[4] A. Mukherjee, V. Venkataraman, B. Liu, and N. Glance, "What Yelp fake review filter might be doing?" in Proc. 7th Int. AAAI Conf. Weblogs and Social Media (ICWSM), Cambridge, MA, USA, 2013, pp. 409–418.

[5] "ReviewMeta: Analyze Amazon product reviews," ReviewMeta. [Online]. Available: https://reviewmeta.com

[6] "Fakespot," Mozilla. [Online]. Available: https://www.fakespot.com (service discontinued in 2025).

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[8] D. Jurafsky and J. H. Martin, Speech and Language Processing, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.

[9] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM), Ann Arbor, MI, USA, 2014, pp. 216–225.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[11] "Time complexity," Python Wiki, Python Software Foundation. [Online]. Available: https://wiki.python.org/moin/TimeComplexity

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[13] "Streamlit documentation," Snowflake Inc. [Online]. Available: https://docs.streamlit.io

[14] "RapidOCR: Awesome OCR multiple programming languages toolkits based on ONNXRuntime, OpenVINO, PaddlePaddle and PyTorch," RapidAI, GitHub. [Online]. Available: https://github.com/RapidAI/RapidOCR

[15] Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.

[16] "Plotly Python graphing library," Plotly Technologies Inc. [Online]. Available: https://plotly.com/python/

[17] "Streamlit Community Cloud," Snowflake Inc. [Online]. Available: https://docs.streamlit.io/deploy/streamlit-community-cloud

APPENDICES

Appendix A: Source Code

The complete source code of the project is listed below. It is also available in the GitHub repository https://github.com/rahulpandey4950/fake-review-detector together with the trained model, the dataset and the requirements file.

File: detector/__init__.py *(Detector package interface)*

```
from .analyzer import Analysis, analyze

__all__ = ["Analysis", "analyze"]
```

File: detector/analyzer.py *(Analyser module)*

```
"""Combines the ML model and the rule-based signals into one
     verdict."""

from __future__ import annotations

from dataclasses import dataclass

from . import heuristics, model

FAKE_THRESHOLD = 0.5
MIN_WORDS = 3


@dataclass
class Analysis:
    text: str
    fake_probability: float
    ml_probability: float | None
    rule_score: float
    signals: list[heuristics.Signal]
    threshold: float = FAKE_THRESHOLD

    @property
    def is_fake(self) -> bool:
        return self.fake_probability >= self.threshold

    @property
    def verdict(self) -> str:
        return "FAKE" if self.is_fake else "REAL"

    @property
    def confidence(self) -> float:
        """How far the score sits past the cutoff toward its end of
     the scale, as 50%-100%.

        Measured against the active threshold, so a stricter or
     looser setting never
```

```
        yields a verdict backed by under 50% confidence.
        """
        p, t = self.fake_probability, self.threshold
        margin = (p - t) / (1 - t) if self.is_fake else (t - p) / t
        return 0.5 + 0.5 * margin

    @property
    def low_confidence(self) -> bool:
        return abs(self.fake_probability - self.threshold) < 0.1


def analyze(text: str, rating: int | None = None) -> Analysis:
    text = text.strip()
    if len(heuristics.WORD_RE.findall(text)) < MIN_WORDS:
        raise ValueError(f"Please provide at least {MIN_WORDS} words
     of review text.")

    signals = heuristics.run_signals(text, rating)
    rule_score = heuristics.combine(signals)
    ml_prob = model.fake_probability(text)
    # The two detectors catch different fakes: the model spots
     machine-written text,
    # the rules spot human-written hype/spam. Either one being
     confident is enough.
    final = rule_score if ml_prob is None else max(ml_prob,
     rule_score)
    return Analysis(text, final, ml_prob, rule_score, signals)
```

File: detector/heuristics.py *(Rule checks module)*

```
"""Rule-based signals for spotting fake reviews.

Each signal checks one well-known trait of fake, paid, or bot-written
     reviews and
returns a score from 0 (looks genuine) to 1 (looks fake) with a
     plain-English
reason. Signals are deliberately simple so every verdict can be
     explained.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

_vader = SentimentIntensityAnalyzer()
# VADER is tuned for social media; nudge it for product reviews
     ("customer support" is
# neutral, "broke" and "defective" are complaints).
_vader.lexicon.update({
    "support": 0.0, "broke": -2.0, "broken": -2.0, "defective": -2.5,
     "faulty": -2.0,
    "refund": -1.0, "returned": -1.0, "stopped": -1.2, "flimsy": -
     1.8, "cheaply": -1.5,
    "overpriced": -1.8, "leaks": -1.5, "leaked": -1.5,
})
```

```
WORD_RE = re.compile(r"[A-Za-z']+")
SENTENCE_SPLIT_RE = re.compile(r"(?<=[.!?])\s+|\n+")

HYPE_WORDS = {
    "amazing", "awesome", "perfect", "perfectly", "excellent",
     "incredible", "fantastic",
    "outstanding", "wonderful", "superb", "flawless", "unbelievable",
     "phenomenal",
    "magnificent", "spectacular", "best", "greatest", "brilliant",
     "extraordinary",
    "terrible", "worst", "horrible", "awful", "garbage", "scam",
     "useless", "pathetic",
    "disgusting", "trash", "fraud",
}

GENERIC_PHRASES = [
    r"highly recommend(ed)?", r"must[- ]buy", r"must[- ]have", r"best
     \w+ ever",
    r"five stars?", r"5 stars?", r"worth every penny", r"you won'?t
     regret",
    r"buy (it|this|one) now", r"don'?t waste your money", r"waste of
     money",
    r"exceeded (all )?my expectations", r"best purchase", r"changed
     my life",
    r"love (it|this)", r"great product", r"works great", r"very good
     product",
    r"value for money", r"go for it", r"totally recommend", r"10/10",
     r"game[- ]changer",
    r"do not buy", r"stay away", r"would recommend (it|this) to
     (everyone|anyone)",
    r"(simply|just) the best", r"nothing (bad|negative) to say",
]
GENERIC_RE = re.compile(r"\b(" + "|".join(GENERIC_PHRASES) + r")\b",
     re.IGNORECASE)

FIRST_PERSON = {"i", "me", "my", "mine", "myself", "i'm", "i've",
     "i'd", "i'll"}

ASPECT_WORDS = {
    "battery", "size", "fit", "fits", "price", "delivery",
     "shipping", "packaging",
    "package", "screen", "sound", "material", "fabric", "color",
     "colour", "smell",
    "taste", "weight", "setup", "install", "installation", "service",
     "refund", "return",
    "returned", "warranty", "instructions", "manual", "button",
     "buttons", "cable",
    "charger", "charging", "stitching", "zipper", "strap", "handle",
     "lid", "box",
    "seller", "customer", "support", "app", "bluetooth", "wifi",
     "noise", "speed",
    "texture", "flavor", "flavour", "plastic", "metal", "leather",
     "wood", "glass",
    "width", "length", "height", "inch", "inches", "cm", "mm", "kg",
     "lbs", "pounds",
    "ounces", "oz", "ml", "liters", "watts", "volume", "temperature",
     "durability",
}
TIME_RE = re.compile(
```

```
    r"\b(\d+|a|one|two|three|few|couple
     of)\s+(minute|hour|day|week|month|year)s?\b"
    r"|\b(yesterday|last (week|month|year)|after using|so far)\b",
    re.IGNORECASE,
)
CONTRAST_RE = re.compile(
    r"\b(but|however|although|though|except|only
     (downside|issue|problem|complaint)"
    r"|wish|cons?|on the other hand|that said|unfortunately)\b",
    re.IGNORECASE,
)
PROMO_RE = re.compile(
    # Phone numbers need a "+" or separators, so ISBNs and model
     numbers don't match.
    r"https?://\S+|www\.\S+|\b[\w.+-]+@[\w-]+\.[\w.]+\b|\+\d[\d\s-
     ]{8,}\d|\b\d{3}[\s.-]\d{3}[\s.-]\d{4}\b"
    r"|\b(coupon|promo code|discount code|use (my|the) code|use my
     link|click (here|the link)"
    r"|visit (our|my)|dm me|whatsapp|telegram|follow me|order
     now|limited offer)\b",
    re.IGNORECASE,
)
INCENTIVE_RE = re.compile(
    r"\b(received (this|the|it)( \w+)? (for free|free|at a
     discount|at a discounted price)"
    r"|in exchange for (my|an) (honest|unbiased|fair)
     (review|opinion)"
    r"|provided (to me )?(by|from) the
     (seller|manufacturer|company|brand)"
    r"|free (sample|product) (for|in return)|sent (me )?(this|it)
     (for free|to review))\b",
    re.IGNORECASE,
)


@dataclass
class Signal:
    key: str
    name: str
    score: float          # 0 = genuine-looking, 1 = fake-looking
    weight: float
    detail: str
    # Evidence-only signals (spam links, rating mismatch...) say
     nothing about a
    # review when absent, so they only join the average when they
     fire.
    evidence_only: bool = False
    spans: list[tuple[int, int]] = field(default_factory=list)

    @property
    def triggered(self) -> bool:
        return self.score >= 0.5

    @property
    def counts(self) -> bool:
        return not self.evidence_only or self.score > 0


def _clamp(x: float) -> float:
    return max(0.0, min(1.0, x))
```

```
def _sentences(text: str) -> list[str]:
    return [s for s in SENTENCE_SPLIT_RE.split(text.strip()) if
     len(WORD_RE.findall(s)) >= 2]


def _sentence_tones(text: str) -> list[float]:
    # Score sentence by sentence: on a whole review VADER lets a
     "never" in one
    # sentence negate "terrible" in the next.
    return [tone for _, tone in sentence_breakdown(text)]


def sentence_breakdown(text: str) -> list[tuple[str, float]]:
    """Each sentence with its VADER tone (-1 negative ... +1
     positive)."""
    parts = [s.strip() for s in SENTENCE_SPLIT_RE.split(text.strip())
     if WORD_RE.search(s)] or [text]
    return [(s, _vader.polarity_scores(s)["compound"]) for s in
     parts]


def _tone(text: str) -> float:
    tones = _sentence_tones(text)
    return sum(tones) / len(tones)


def hype_words(text: str, words: list[re.Match]) -> Signal:
    hits = [m for m in words if m.group().lower() in HYPE_WORDS]
    density = len(hits) / max(1, len(words))
    score = _clamp(density / 0.08)
    return Signal(
        "hype", "Extreme / superlative words", score, 1.2,
        f"{len(hits)} extreme word(s) ({density:.0%} of the text). "
        "Fake reviews lean on words like 'amazing', 'perfect' or
     'worst'.",
        spans=[m.span() for m in hits],
    )


def generic_phrases(text: str, words: list[re.Match]) -> Signal:
    hits = list(GENERIC_RE.finditer(text))
    score = _clamp(0.35 * len(hits))
    return Signal(
        "generic", "Stock / template phrases", score, 1.2,
        f"{len(hits)} stock phrase(s) such as 'highly recommend' or
     'worth every penny'. "
        "Paid reviews often reuse the same template lines.",
        spans=[m.span() for m in hits],
    )


def shouting(text: str, words: list[re.Match]) -> Signal:
    exclaims = text.count("!")
    caps = [m for m in words if len(m.group()) >= 3 and
     m.group().isupper()]
    sentences = max(1, len(_sentences(text)))
```

```
    score = _clamp(0.4 * exclaims / sentences + 4 * len(caps) /
     max(1, len(words))
                   + (0.3 if re.search(r"[!?]{2,}", text) else 0))
    return Signal(
        "shouting", "Excessive punctuation / CAPS", score, 0.8,
        f"{exclaims} exclamation mark(s) and {len(caps)} ALL-CAPS
     word(s). "
        "Over-the-top emphasis is common in exaggerated reviews.",
        spans=[m.span() for m in caps],
    )


def first_person(text: str, words: list[re.Match]) -> Signal:
    count = sum(1 for m in words if m.group().lower() in
     FIRST_PERSON)
    ratio = count / max(1, len(words))
    score = _clamp((ratio - 0.07) / 0.08)
    return Signal(
        "first_person", "Self-focused language", score, 0.5,
        f"'I / me / my' make up {ratio:.0%} of words. Research on
     deceptive reviews "
        "(Ott et al., 2011) found fakes talk about the writer more
     than the product.",
    )


def lack_of_specifics(text: str, words: list[re.Match]) -> Signal:
    aspects = [m for m in words if m.group().lower() in ASPECT_WORDS]
    numbers = re.findall(r"\b\d+(\.\d+)?\b", text)
    times = TIME_RE.findall(text)
    found = len(aspects) + len(numbers) + len(times)
    expected = max(1.0, len(words) / 25)
    score = _clamp(1 - found / expected)
    return Signal(
        "specifics", "Lack of concrete details", score, 1.2,
        f"{found} concrete detail(s) (product features, numbers, how
     long it was used). "
        "Genuine buyers usually mention specifics; fakes stay
     vague.",
    )


def one_sided(text: str, words: list[re.Match]) -> Signal:
    scores = _sentence_tones(text)
    overall = sum(scores) / len(scores)
    has_pos = any(s > 0.3 for s in scores)
    has_neg = any(s < -0.3 for s in scores)
    mixed = (has_pos and has_neg) or bool(CONTRAST_RE.search(text))
    score = abs(overall) * (0.3 if mixed else 1.0)
    tone = "positive" if overall > 0 else "negative"
    detail = (
        "Mentions both pros and cons, which is typical of real
     reviews."
        if mixed else
        f"Entirely {tone} with no trade-offs (sentiment
     {overall:+.2f}). "
        "Real experiences are rarely 100% one-sided."
    )
    return Signal("one_sided", "One-sided sentiment", score, 1.0,
     detail)
```

```
def too_short(text: str, words: list[re.Match]) -> Signal:
    n = len(words)
    score = 0.9 if n < 8 else 0.6 if n < 20 else 0.3 if n < 40 else
     0.1
    return Signal(
        "length", "Very short review", score, 0.7,
        f"{n} words. Very short reviews give little evidence of real
     use.",
    )


def repetition(text: str, words: list[re.Match]) -> Signal:
    tokens = [m.group().lower() for m in words][:120]
    if len(tokens) < 15:
        return Signal("repetition", "Repetitive wording", 0.0, 0.8,
                      "Too short to judge repetition.",
     evidence_only=True)
    ttr = len(set(tokens)) / len(tokens)
    bigrams = list(zip(tokens, tokens[1:]))
    repeated = len(bigrams) - len(set(bigrams))
    score = _clamp((0.6 - ttr) / 0.25 + repeated / 10)
    return Signal(
        "repetition", "Repetitive wording", score, 0.8,
        f"Vocabulary variety {ttr:.0%}, {repeated} repeated word
     pair(s). "
        "Bots and copy-paste reviews repeat themselves.",
    )


def promotional(text: str, words: list[re.Match]) -> Signal:
    hits = list(PROMO_RE.finditer(text))
    score = 0.0 if not hits else 0.75 if len(hits) == 1 else 0.95
    return Signal(
        "promo", "Links / promo codes / contact info", score, 1.5,
        f"{len(hits)} link(s), code(s) or contact detail(s) found.
     Reviews that advertise are spam."
        if hits else "No links, promo codes or contact details.",
        evidence_only=True, spans=[m.span() for m in hits],
    )


def incentivized(text: str, words: list[re.Match]) -> Signal:
    hits = list(INCENTIVE_RE.finditer(text))
    return Signal(
        "incentive", "Incentivized review disclosure", 0.65 if hits
     else 0.0, 1.0,
        "Says the product was free or discounted in exchange for a
     review, so it is not independent."
        if hits else "No sign the reviewer was paid or given the
     product.",
        evidence_only=True, spans=[m.span() for m in hits],
    )


def rating_mismatch(text: str, rating: int | None) -> Signal:
    if rating is None:
```

```
        return Signal("rating", "Star rating vs. text mismatch", 0.0,
     1.5,
                      "No star rating given.", evidence_only=True)
    compound = _tone(text)
    mismatch = (rating >= 4 and compound < -0.3) or (rating <= 2 and
     compound > 0.4)
    return Signal(
        "rating", "Star rating vs. text mismatch", 0.9 if mismatch
     else 0.0, 1.5,
        f"{rating}-star rating but the text reads {'negative' if
     compound < 0 else 'positive'} "
        f"(sentiment {compound:+.2f})." if mismatch else
        f"{rating}-star rating is consistent with the tone of the
     text.",
        evidence_only=True,
    )


TEXT_SIGNALS = [
    hype_words, generic_phrases, shouting, first_person,
     lack_of_specifics,
    one_sided, too_short, repetition, promotional, incentivized,
]


def run_signals(text: str, rating: int | None = None) ->
     list[Signal]:
    words = list(WORD_RE.finditer(text))
    signals = [fn(text, words) for fn in TEXT_SIGNALS]
    signals.append(rating_mismatch(text, rating))
    return signals


def combine(signals: list[Signal]) -> float:
    """Weighted average of the signals that carry evidence, floored
     by the strongest
    evidence-only signal: a promo link or a rating mismatch is
     damning on its own and
    must not be averaged away by the checks that found nothing."""
    active = [s for s in signals if s.counts]
    total = sum(s.weight for s in active)
    average = sum(s.score * s.weight for s in active) / total if
     total else 0.0
    strongest = max((s.score for s in signals if s.evidence_only),
     default=0.0)
    return max(average, strongest)
```

File: detector/model.py *(Model loading module)*

```
"""Machine-learning half of the detector: a TF-IDF + logistic
     regression classifier.

Trained by ``train_model.py`` on the public Fake Reviews Dataset
     (Salminen et al.,
2022: 20k genuine Amazon reviews vs 20k computer-generated ones).
"""

from __future__ import annotations
```

```
from functools import lru_cache
from pathlib import Path

import joblib

MODEL_PATH = Path(__file__).resolve().parent.parent / "models" /
     "review_model.joblib"


@lru_cache(maxsize=1)
def load_model() -> dict | None:
    """Return the saved bundle ``{"pipeline", "metrics", ...}`` or
     None if not trained yet."""
    if not MODEL_PATH.exists():
        return None
    return joblib.load(MODEL_PATH)


def fake_probability(text: str) -> float | None:
    bundle = load_model()
    if bundle is None:
        return None
    pipeline = bundle["pipeline"]
    fake_index = list(pipeline.classes_).index(1)
    return float(pipeline.predict_proba([text])[0][fake_index])
```

File: detector/ocr.py *(OCR module)*

```
"""Pull review text out of a screenshot using RapidOCR (ONNX models
     ship inside the pip
package, so there is nothing to install on the server and nothing
     downloaded at runtime)."""

from __future__ import annotations

import re
from dataclasses import dataclass
from functools import lru_cache

import numpy as np
from PIL import Image

# Screenshot chrome from shopping sites that is not part of the
     review itself.
NOISE_RE = re.compile(
    r"^(helpful|report( abuse)?|verified purchase|read more|see
     more|show more|translate( review)?"
    r"|was this (review )?helpful\??|yes|no|like|reply|share|top
     reviews?.*|customer reviews?"
    r"|\d+ (people|person) found this helpful|reviewed in .* on
     .*|\d[\d,]* (ratings?|reviews?))$",
    re.IGNORECASE,
)
NOISE_WORDS = {"helpful", "report", "abuse", "like", "reply",
     "share", "translate", "yes", "no", "|"}
RATING_RE = re.compile(r"\b([1-5](?:\.\d)?)\s*(?:out of
     5|/\s*5)(?:\s*stars?)?", re.IGNORECASE)
VERIFIED_RE = re.compile(r"verified (purchase|buyer)", re.IGNORECASE)
```

```
# The recogniser occasionally garbles a line at one scale and reads
     it perfectly at
# another, with a low confidence score when it fails, so we try a few
     scales and keep
# the most confident read.
SCALES = (1.0, 1.5, 2.0)
MAX_SIDE = 4000
GOOD_ENOUGH = 0.93


@dataclass
class OcrResult:
    text: str
    rating: int | None
    verified_badge: bool
    lines: list[str]


@lru_cache(maxsize=1)
def _engine():
    from rapidocr import RapidOCR
    return RapidOCR(params={"Global.log_level": "warning"})


def _best_read(image: Image.Image):
    """OCR the image at a few scales and return the output with the
     highest mean confidence."""
    image = image.convert("RGB")
    best, best_score = None, -1.0
    for scale in SCALES:
        if scale > 1 and max(image.size) * scale > MAX_SIDE:
            break
        scaled = image if scale == 1 else image.resize(
            (round(image.width * scale), round(image.height *
     scale)), Image.LANCZOS)
        out = _engine()(np.array(scaled))
        if out.boxes is None:
            continue
        # Weight by text length so one confident short word can't
     outvote a garbled paragraph.
        score = sum(s * len(t) for s, t in zip(out.scores, out.txts))
     / max(1, sum(map(len, out.txts)))
        if score > best_score:
            best, best_score = out, score
        if score >= GOOD_ENOUGH and min(out.scores) >= 0.8:
            break
    return best


def _group_lines(boxes, texts) -> list[str]:
    """Merge OCR fragments that sit on the same visual row, then
     order rows top-to-bottom."""
    items = []
    for box, text in zip(boxes, texts):
        ys = [p[1] for p in box]
        items.append((min(ys), max(ys), min(p[0] for p in box),
     text))
    items.sort()
    rows: list[list] = []
    for top, bottom, left, text in items:
```

```
        center = (top + bottom) / 2
        if rows and abs(center - rows[-1][0]) < (bottom - top) / 2:
            rows[-1][1].append((left, text))
        else:
            rows.append([center, [(left, text)]])
    return [" ".join(t for _, t in sorted(parts)) for _, parts in
     rows]


def extract_review(image: Image.Image) -> OcrResult:
    out = _best_read(image)
    lines = [] if out is None else _group_lines(out.boxes.tolist(),
     out.txts)
    full = "\n".join(lines)

    rating_match = RATING_RE.search(full)
    rating = round(float(rating_match.group(1))) if rating_match else
     None
    review_lines = []
    for line in lines:
        line = RATING_RE.sub("", line).strip()   # "5.0 out of 5
     stars Great!" keeps "Great!"
        if line and not NOISE_RE.match(line) and not
     set(line.lower().split()) <= NOISE_WORDS:
            review_lines.append(line)
    return OcrResult(
        text=" ".join(review_lines).strip(),
        rating=rating,
        verified_badge=bool(VERIFIED_RE.search(full)),
        lines=lines,
    )
```

File: app.py *(User interface (main page))*

```
"""Fake Review Detector - Streamlit front end.

Run locally:  streamlit run app.py
"""

from __future__ import annotations

import dataclasses
import hashlib
import io
from datetime import datetime

import pandas as pd
import streamlit as st
from PIL import Image

from detector import Analysis, analyze
from detector.model import load_model
from ui import components as ui


st.set_page_config(page_title="Fake Review Detector", page_icon="🕵️",
     layout="centered")

EXAMPLES = {
```

```
    "😬 Suspicious": (
        "AMAZING product!!! Best purchase ever, highly recommend to
     everyone. "
        "Absolutely perfect, five stars!!! You won't regret it! Buy
     it now!"
    ),
    "😳 Genuine": (
        "I've been using this kettle for about three months. It boils
     1.5 liters in roughly "
        "four minutes, which is faster than my old one. The lid is a
     bit stiff to open and the "
        "cable is short, but for the price it does the job well."
    ),
    "📢 Spam": (
        "Great product, works great. Use my code SAVE20 at www.dealz-
     shop.com for a discount!"
    ),
}
RATING_OPTIONS = ["Not given", 1, 2, 3, 4, 5]
# Strictness level -> fake-score cutoff. Stricter means a lower
     cutoff, so more reviews get flagged.
STRICTNESS = {"Relaxed": 0.65, "Balanced": 0.50, "Strict": 0.35}

state = st.session_state
state.setdefault("results", {})      # tab -> (Analysis,
     verified_badge)
state.setdefault("history", [])
state.setdefault("auto_run", False)


@st.cache_resource(show_spinner="Loading OCR engine...")
def get_ocr():
    from detector import ocr
    return ocr


@st.cache_data(show_spinner="Reading text from image...",
     max_entries=32)
def run_ocr(image_bytes: bytes):
    image = Image.open(io.BytesIO(image_bytes))
    return get_ocr().extract_review(image)


def run_analysis(tab: str, text: str, rating, verified_badge: bool =
     False) -> None:
    try:
        result = analyze(text, None if rating == "Not given" else
     int(rating))
    except ValueError as exc:
        st.warning(str(exc))
        return
    state.results[tab] = (result, verified_badge)
    state.history.insert(0, {
        "Time": datetime.now().strftime("%H:%M:%S"),
        "Source": "🖼️ Screenshot" if tab == "image" else "✍️ Text",
        "Review": text[:70] + ("…" if len(text) > 70 else ""),
        "score": result.fake_probability,
    })
    st.toast("Analysis complete", icon="🔎")
```

```
def show_result(tab: str, threshold: float) -> None:
    if tab not in state.results:
        return
    analysis, verified_badge = state.results[tab]
    # Re-apply the sidebar threshold so moving the slider updates the
     verdict live.
    a: Analysis = dataclasses.replace(analysis, threshold=threshold)
    ui.verdict_card(a)
    ui.stat_tiles(a)
    if verified_badge:
        st.info("**Verified Purchase** badge spotted in the
     screenshot: a mild sign the reviewer bought it.",
                icon="🛒")

    hl, crit, mood, details = st.tabs(["🖍️ Highlights", "📊
     Criteria", "💬 Sentence mood", "📋 Details"])
    with hl:
        ui.highlighted_review(a)
    with crit:
        ui.criteria_chart(a)
    with mood:
        ui.sentence_chart(a)
    with details:
        ui.criteria_list(a)


def use_example(name: str) -> None:
    state.review_text = EXAMPLES[name]
    state.auto_run = True


# ---------------------------------------------------------------------
     sidebar
with st.sidebar:
    st.markdown("### 🎚️ Strictness")
    strictness = st.select_slider(
        "How strict should the detector be?",
     options=list(STRICTNESS), value="Balanced",
        help="Strict flags more reviews as fake; Relaxed flags only
     clear-cut cases. "
             "The verdict updates instantly.",
    )
    threshold = STRICTNESS[strictness]
    st.caption(f"A review is flagged as fake when its score is
     **{threshold:.0%} or more**.")
    st.markdown("### 🧠 How it works")
    st.markdown(
        "Two detectors check every review. It's flagged if **either**
     is confident:\n\n"
        "1. **🤖 ML model**: spots machine-generated text (TF-IDF +
     logistic regression).\n"
        "2. **🪚 Rule checks**: hype words, stock phrases, missing
     details, one-sided tone, "
        "CAPS/!!!, spam links, paid-review disclosures, star rating
     vs. text."
    )
```

```
    bundle = load_model()
    if bundle:
        m = bundle["metrics"]
        c1, c2 = st.columns(2)
        c1.metric("Accuracy", f"{m['accuracy']:.1%}")
        c2.metric("ROC AUC", f"{m['roc_auc']:.3f}")
        st.caption(f"Held-out test set · trained on
     {bundle['n_reviews']:,} reviews "
                   f"({bundle['dataset']}).")
    else:
        st.warning("ML model not found: using rule checks only. Run
     `python train_model.py`.")
    st.divider()
    st.caption("⚠️ Showcase project. Scores are probabilistic and can
     be wrong; don't use them as "
               "the only basis for decisions about real people or
     businesses.")

# ---------------------------------------------------------------------
     main
ui.load_css()
ui.hero()

text_tab, image_tab = st.tabs(["✍️ Paste review", "🖼️ Upload
     screenshot"])

with text_tab:
    st.caption("Try an example:")
    cols = st.columns(len(EXAMPLES))
    for col, name in zip(cols, EXAMPLES):
        col.button(name, on_click=use_example, args=(name,),
     width="stretch")
    text = st.text_area("Review text", key="review_text", height=150,
                        placeholder="Paste the review here...")
    c1, c2 = st.columns([2, 1], vertical_alignment="bottom")
    rating = c1.selectbox("Star rating (optional)", RATING_OPTIONS,
     key="text_rating",
                          help="Lets the detector check whether the
     stars match the tone of the text.")
    clicked = c2.button("🔍 Analyze review", type="primary",
     key="analyze_text", width="stretch")
    if clicked or state.auto_run:
        state.auto_run = False
        run_analysis("text", text, rating)
    show_result("text", threshold)

with image_tab:
    upload = st.file_uploader("Screenshot of a review", type=["png",
     "jpg", "jpeg", "webp"])
    if upload is None:
        state.results.pop("image", None)
    else:
        image_bytes = upload.getvalue()
        with st.expander("🖼️ Uploaded screenshot", expanded=True):
            st.image(image_bytes, width="stretch")
        ocr_result = run_ocr(image_bytes)
        if not ocr_result.text:
            st.warning("No readable text found in this image. Try a
     sharper or larger screenshot.")
        else:
```

```
            # Key on the file hash so a new upload refreshes the
    editable text.
            key = hashlib.md5(image_bytes).hexdigest()[:10]
            if state.get("image_key") != key:
                state.image_key = key
                state.results.pop("image", None)
            ocr_text = st.text_area("Extracted text (edit if the OCR
    got something wrong)",
                                    value=ocr_result.text,
    height=150, key=f"ocr_{key}")
            default = RATING_OPTIONS.index(ocr_result.rating) if
    ocr_result.rating else 0
            c1, c2 = st.columns([2, 1], vertical_alignment="bottom")
            ocr_rating = c1.selectbox("Star rating", RATING_OPTIONS,
    index=default, key=f"rating_{key}",
                                      help="Detected automatically
    when the screenshot shows 'x out of 5'.")
            if c2.button("🔍 Analyze screenshot", type="primary",
    key="analyze_image", width="stretch"):
                run_analysis("image", ocr_text, ocr_rating,
    ocr_result.verified_badge)
            show_result("image", threshold)

# ---------------------------------------------------------------
    history
if state.history:
    st.markdown("### 🗂️ Your checks this session")
    df = pd.DataFrame(state.history)
    df["Verdict"] = ["🚩 Fake" if s >= threshold else "✅ Real" for
     s in df["score"]]
    df["Fake score"] = df["score"] * 100
    st.dataframe(
        df[["Time", "Verdict", "Fake score", "Source", "Review"]],
     hide_index=True, width="stretch",
        column_config={"Fake score": st.column_config.ProgressColumn(
            min_value=0, max_value=100, format="%.0f%%")},
    )
    if st.button("🧹 Clear history"):
        state.history.clear()
        st.rerun()
```

File: ui/components.py *(User interface components)*

```
"""Rendering helpers for the Streamlit page: styled cards, highlights
    and charts."""

from __future__ import annotations

import html
from pathlib import Path

import plotly.graph_objects as go
import streamlit as st

from detector import Analysis
from detector.heuristics import WORD_RE, sentence_breakdown

CSS_PATH = Path(__file__).with_name("style.css")
```

```
# Status palette (state, never reused for series) - always paired
    with an icon + label.
GOOD, WARNING, CRITICAL = "#0ca30c", "#fab219", "#d03b3b"
# Diverging pair for sentence tone: red (negative) <-> gray midpoint
    <-> blue (positive).
NEGATIVE, NEUTRAL, POSITIVE = "#e34948", "#a8a7a1", "#2a78d6"

# Criteria whose matched phrases are highlighted in the review, with
    their legend label.
HIGHLIGHTS = {
    "hype": "Extreme words",
    "generic": "Stock phrases",
    "shouting": "CAPS",
    "promo": "Links / codes",
    "incentive": "Incentive",
}


def load_css() -> None:
    st.markdown(f"<style>{CSS_PATH.read_text(encoding='utf-
    8')}</style>", unsafe_allow_html=True)


def hero() -> None:
    st.markdown(
        """
        <div class="hero">
          <h1>🕵️‍♀️ Fake Review Detector</h1>
          <p>Paste a product review or drop in a screenshot. We'll
    tell you if it smells fishy, and why.</p>
          <div class="chips">
            <span class="chip">🤖 ML model · 95% accurate</span>
            <span class="chip">🔪 11 explainable checks</span>
            <span class="chip">🖼️ Screenshot OCR</span>
          </div>
        </div>
        """,
        unsafe_allow_html=True,
    )


def _status(score: float) -> tuple[str, str, str]:
    """(icon, label, color) for a 0-1 criterion score."""
    if score >= 0.5:
        return "🚩", "Red flag", CRITICAL
    if score >= 0.3:
        return "⚠️", "Watch", WARNING
    return "✅", "OK", GOOD


def verdict_card(a: Analysis) -> None:
    kind = "fake" if a.is_fake else "real"
    title = "🚩 Likely FAKE" if a.is_fake else "✅ Looks REAL"
    sub = f"{a.confidence:.0%} confidence"
    if a.low_confidence:
        sub += " · borderline, treat as a hint"
    pct = a.fake_probability * 100
    st.markdown(
```

```
        f"""
        <div class="verdict {kind}">
          <div class="row">
            <div>
              <div class="label">Verdict</div>
              <div class="title">{title}</div>
              <div class="sub">{sub}</div>
            </div>
            <div class="hero-num">{pct:.0f}%<small>FAKE
     SCORE</small></div>
          </div>
          <div class="meter" role="meter" aria-valuenow="{pct:.0f}"
     aria-valuemin="0" aria-valuemax="100"
               aria-label="Fake score">
            <div class="fill" style="width:{max(pct, 2):.1f}%"></div>
            <div class="tick" style="left:{a.threshold * 100:.0f}%"
     title="Fake threshold"></div>
          </div>
          <div class="meter-scale"><span>0% genuine</span>
            <span>threshold {a.threshold:.0%}</span><span>100%
     fake</span></div>
        </div>
        """,
        unsafe_allow_html=True,
    )


def stat_tiles(a: Analysis) -> None:
    words = len(WORD_RE.findall(a.text))
    tones = [t for _, t in sentence_breakdown(a.text)]
    tone = sum(tones) / len(tones)
    flags = sum(1 for s in a.signals if s.counts and s.triggered)
    ml = "n/a" if a.ml_probability is None else
     f"{a.ml_probability:.0%}"
    tiles = [
        ("ML model", ml, "machine-written text", "#4f46e5"),
        ("Rule checks", f"{a.rule_score:.0%}", "human hype & spam",
     "#9333ea"),
        ("Red flags", f"{flags}", f"of {sum(1 for s in a.signals if
     s.counts)} checks", "#ec4899"),
        ("Tone", f"{tone:+.2f}", f"{words} words", "#f97316"),
    ]
    st.markdown(
        '<div class="tiles">' + "".join(
            f'<div class="tile" style="--t-color:{c}"><div class="t-
     label">{label}</div>'
            f'<div class="t-value">{value}</div><div class="t-
     note">{note}</div></div>'
            for label, value, note, c in tiles
        ) + "</div>",
        unsafe_allow_html=True,
    )
    if a.ml_probability is not None:
        driver = "ML model" if a.ml_probability >= a.rule_score else
     "rule checks"
        st.caption(f"The fake score is the higher of the two
     detectors (here: the {driver}).")


def highlighted_review(a: Analysis) -> None:
```

```
    """The review with each triggering phrase marked in its
     criterion's color."""
    spans = sorted(
        (start, end, s.key, s.name)
        for s in a.signals if s.triggered and s.key in HIGHLIGHTS
        for start, end in s.spans
    )
    out, pos, used = [], 0, []
    for start, end, key, name in spans:
        if start < pos:
            continue  # overlaps a phrase already marked
        out.append(html.escape(a.text[pos:start]))
        out.append(f'<mark class="m-{key}"
     title="{html.escape(name)}">{html.escape(a.text[start:end])}</ma
     rk>')
        pos = end
        if key not in used:
            used.append(key)
    out.append(html.escape(a.text[pos:]))
    body = "".join(out).replace("\n", "<br>")

    legend = "".join(f'<span class="m-
     {k}"><i></i>{HIGHLIGHTS[k]}</span>' for k in HIGHLIGHTS if k in
     used)
    st.markdown(f'<div class="review-box">{body}</div>',
     unsafe_allow_html=True)
    if legend:
        st.markdown(f'<div class="legend">{legend}</div>',
     unsafe_allow_html=True)
        st.caption("Hover a highlighted phrase to see which check it
     tripped.")
    else:
        st.caption("Nothing in the wording tripped a phrase-level
     check.")


def _base_layout(fig: go.Figure, height: int) -> go.Figure:
    fig.update_layout(
        height=height,
        margin=dict(l=8, r=16, t=8, b=8),
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        showlegend=False,
        hoverlabel=dict(align="left"),
        font=dict(size=13),
    )
    return fig


def criteria_chart(a: Analysis) -> None:
    rows = sorted((s for s in a.signals if s.counts), key=lambda s:
     s.score)
    labels = [f"{_status(s.score)[0]} {s.name}" for s in rows]
    colors = [_status(s.score)[2] for s in rows]
    hover = [
        f"<b>{html.escape(s.name)}</b><br>{s.score:.0%} ·
     {_status(s.score)[1]} · weight {s.weight}"
        f"<br><br>{'<br>'.join(_wrap(html.escape(s.detail), 60))}"
        for s in rows
    ]
```

```
    fig = go.Figure(go.Bar(
        x=[s.score * 100 for s in rows], y=labels, orientation="h",
        marker=dict(color=colors, cornerradius=4),
        text=[f"{s.score:.0%}" for s in rows],
     textposition="outside", cliponaxis=False,
        hovertext=hover, hoverinfo="text",
    ))
    fig.add_vline(x=50, line=dict(color="rgba(127,127,127,.6)",
     width=1, dash="dot"),
                  annotation_text="red-flag line",
     annotation_position="top", annotation_font_size=11)
    fig.update_xaxes(range=[0, 112], ticksuffix="%",
     gridcolor="rgba(127,127,127,.15)", zeroline=False)
    fig.update_yaxes(showgrid=False)
    fig = _base_layout(fig, 70 + 34 * len(rows))
    fig.update_layout(bargap=0.35, margin=dict(t=28))
    st.plotly_chart(fig, config={"displayModeBar": False},
                    theme="streamlit")
    st.caption("✅ OK (under 30%) · ⚠️ Watch (30-49%) · 🚩 Red flag
     (50%+). Hover a bar for the reasoning.")


def sentence_chart(a: Analysis) -> None:
    parts = sentence_breakdown(a.text)
    tones = [t for _, t in parts]
    colors = [POSITIVE if t > 0.05 else NEGATIVE if t < -0.05 else
     NEUTRAL for t in tones]
    labels = [f"S{i}" for i in range(1, len(parts) + 1)]
    hover = [f"<b>{lab}</b> · tone
     {t:+.2f}<br>{'<br>'.join(_wrap(html.escape(s), 60))}"
             for lab, (s, t) in zip(labels, parts)]
    fig = go.Figure(go.Bar(
        x=tones, y=labels, orientation="h",
        marker=dict(color=colors, cornerradius=4),
        # Labels keep neutral (zero-length) sentences visible.
        text=[f"{t:+.2f}" for t in tones], textposition="outside",
     cliponaxis=False,
        hovertext=hover, hoverinfo="text",
    ))
    fig.update_xaxes(range=[-1.15, 1.15], tickvals=[-1, -0.5, 0, 0.5,
     1],
                     ticktext=["−1 negative", "−0.5", "0", "+0.5",
     "+1 positive"],
                     gridcolor="rgba(127,127,127,.15)",
     zeroline=True,
                     zerolinecolor="rgba(127,127,127,.6)",
     zerolinewidth=1)
    fig.update_yaxes(autorange="reversed", showgrid=False)
    fig.update_layout(bargap=0.35)
    st.plotly_chart(_base_layout(fig, 70 + 32 * len(parts)),
     config={"displayModeBar": False},
                    theme="streamlit")
    one_sided = next(s for s in a.signals if s.key == "one_sided")
    st.caption("Each bar is one sentence (hover to read it). Blue =
     positive, red = negative, "
               "gray = neutral. " + one_sided.detail)


def criteria_list(a: Analysis) -> None:
```

```
    rows = sorted((s for s in a.signals if s.counts), key=lambda s:
     s.score * s.weight, reverse=True)
    items = []
    for s in rows:
        icon, label, color = _status(s.score)
        items.append(
            f'<div class="crit"><span>{icon}</span><span class="c-
     name">{html.escape(s.name)}</span>'
            f'<span class="c-badge" style="--b:{color}">{label} ·
     {s.score:.0%}</span>'
            f'<span class="c-
     detail">{html.escape(s.detail)}</span></div>'
        )
    st.markdown("".join(items), unsafe_allow_html=True)
    skipped = [s.name for s in a.signals if not s.counts]
    if skipped:
        st.caption("No evidence either way: " + ", ".join(skipped))


def _wrap(text: str, width: int) -> list[str]:
    lines, line = [], ""
    for word in text.split():
        if line and len(line) + len(word) + 1 > width:
            lines.append(line)
            line = word
        else:
            line = f"{line} {word}".strip()
    return lines + ([line] if line else [])
```

File: ui/style.css *(User interface styling)*

```
/* ---------- page ---------- */
.block-container { padding-top: 2rem; max-width: 860px; }

/* ---------- hero banner ---------- */
.hero {
  position: relative;
  overflow: hidden;
  padding: 28px 28px 24px;
  border-radius: 20px;
  color: #fff;
  background: linear-gradient(120deg, #4f46e5 0%, #9333ea 45%,
     #ec4899 75%, #f97316 100%);
  background-size: 200% 200%;
  animation: hero-shift 12s ease infinite;
  box-shadow: 0 12px 32px rgba(79, 70, 229, .25);
  margin-bottom: 1.2rem;
}
.hero h1 { color: #fff !important; font-size: 2.1rem; margin: 0 0
     6px; padding: 0; }
.hero p { margin: 0; opacity: .92; font-size: 1.02rem; }
.hero .chips { margin-top: 14px; display: flex; flex-wrap: wrap; gap:
     8px; }
.hero .chip {
  background: rgba(255, 255, 255, .18);
  border: 1px solid rgba(255, 255, 255, .35);
  border-radius: 999px;
  padding: 3px 12px;
  font-size: .82rem;
  backdrop-filter: blur(4px);
```

```
}
.hero::after {
  content: "";
  position: absolute; right: -40px; top: -40px;
  width: 180px; height: 180px; border-radius: 50%;
  background: rgba(255, 255, 255, .12);
}
@keyframes hero-shift {
  0%, 100% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
}

/* ---------- tabs ---------- */
.stTabs [role="tablist"] { gap: 6px; }
.stTabs [role="tab"] {
  border-radius: 999px;
  padding: 6px 16px;
  background: rgba(127, 127, 127, .08);
}
.stTabs [role="tab"][aria-selected="true"] {
  background: linear-gradient(120deg, #4f46e5, #9333ea);
  color: #fff;
}
.stTabs [role="tab"][aria-selected="true"] p { color: #fff; }
.stTabs [data-baseweb="tab-highlight"], .stTabs [data-baseweb="tab-
    border"] { display: none; }

/* ---------- primary button ---------- */
.stButton button[kind="primary"] {
  background: linear-gradient(120deg, #4f46e5, #9333ea 60%, #ec4899);
  border: none;
  box-shadow: 0 6px 18px rgba(147, 51, 234, .3);
  transition: transform .15s ease, box-shadow .15s ease;
}
.stButton button[kind="primary"]:hover {
  transform: translateY(-1px);
  box-shadow: 0 10px 24px rgba(147, 51, 234, .4);
}

/* ---------- verdict card ---------- */
.verdict {
  border-radius: 18px;
  padding: 22px 24px 18px;
  margin: 1rem 0 .6rem;
  border: 1px solid var(--v-border);
  background: var(--v-bg);
  animation: pop-in .45s cubic-bezier(.2, .9, .3, 1.2);
}
.verdict.fake { --v-accent: #d03b3b; --v-bg: rgba(208, 59, 59, .10);
    --v-border: rgba(208, 59, 59, .35); }
.verdict.real { --v-accent: #0ca30c; --v-bg: rgba(12, 163, 12, .10);
    --v-border: rgba(12, 163, 12, .35); }
.verdict .row { display: flex; align-items: center; justify-content:
    space-between; gap: 16px; flex-wrap: wrap; }
.verdict .label { font-size: .85rem; text-transform: uppercase;
    letter-spacing: .08em; opacity: .75; }
.verdict .title { font-size: 1.9rem; font-weight: 700; color: var(--
    v-accent); line-height: 1.15; }
.verdict .sub { font-size: .92rem; opacity: .8; margin-top: 2px; }
.verdict .hero-num { font-size: 3.4rem; font-weight: 700; line-
    height: 1; text-align: right; }
```

```
.verdict .hero-num small { display: block; font-size: .8rem; font-
    weight: 500; opacity: .7; letter-spacing: .04em; }

/* meter: fill = severity, track = lighter step of the same hue */
.meter { position: relative; height: 12px; border-radius: 999px;
    margin-top: 16px;
        background: color-mix(in srgb, var(--v-accent) 18%,
    transparent); }
.meter .fill { height: 100%; border-radius: 999px; background: var(--
    v-accent);
              animation: grow 0.9s ease-out; transform-origin: left;
    }
.meter .tick { position: absolute; top: -5px; width: 2px; height:
    22px; background: currentColor; opacity: .55; }
.meter-scale { display: flex; justify-content: space-between; font-
    size: .75rem; opacity: .65; margin-top: 4px; }
@keyframes grow { from { transform: scaleX(0); } to { transform:
    scaleX(1); } }
@keyframes pop-in { from { opacity: 0; transform: translateY(8px)
    scale(.98); } to { opacity: 1; transform: none; } }

/* ---------- stat tiles ---------- */
.tiles { display: grid; grid-template-columns: repeat(4, minmax(0,
    1fr)); gap: 10px; margin: .4rem 0 1rem; }
@media (max-width: 640px) { .tiles { grid-template-columns: repeat(2,
    minmax(0, 1fr)); } }
.tile {
  border-radius: 14px; padding: 12px 14px;
  background: rgba(127, 127, 127, .07);
  border: 1px solid rgba(127, 127, 127, .18);
  border-top: 3px solid var(--t-color, #4f46e5);
}
.tile .t-label { font-size: .78rem; opacity: .7; }
.tile .t-value { font-size: 1.5rem; font-weight: 700; margin-top:
    2px; }
.tile .t-note { font-size: .74rem; opacity: .65; }

/* ---------- highlighted review ---------- */
.review-box {
  padding: 16px 18px; border-radius: 14px; line-height: 1.9; font-
    size: 1.02rem;
  background: rgba(127, 127, 127, .06); border: 1px solid rgba(127,
    127, 127, .2);
}
.review-box mark {
  color: inherit; padding: 1px 3px; border-radius: 4px;
  background: color-mix(in srgb, var(--m) 28%, transparent);
  border-bottom: 2px solid var(--m);
  cursor: help;
}
.legend { display: flex; flex-wrap: wrap; gap: 8px 14px; margin: 10px
    0 2px; font-size: .84rem; }
.legend span { display: inline-flex; align-items: center; gap: 6px; }
.legend i { width: 12px; height: 12px; border-radius: 3px;
    background: var(--m); display: inline-block; }

/* ---------- criteria list ---------- */
.crit { display: grid; grid-template-columns: 26px 1fr auto; gap: 4px
    10px; align-items: center;
```

```
        padding: 10px 4px; border-bottom: 1px solid rgba(127, 127,
    127, .15); }
.crit .c-name { font-weight: 600; }
.crit .c-detail { grid-column: 2 / 4; font-size: .86rem; opacity: .8;
    }
.crit .c-badge { font-size: .76rem; font-weight: 600; padding: 2px
    10px; border-radius: 999px;
                border: 1px solid var(--b); background: color-mix(in
    srgb, var(--b) 18%, transparent); }

/* highlight colors: categorical slots 1-5, stepped for each surface
    */
.m-hype { --m: #2a78d6; } .m-generic { --m: #eb6834; } .m-shouting {
    --m: #1baf7a; }
.m-promo { --m: #eda100; } .m-incentive { --m: #e87ba4; }
@media (prefers-color-scheme: dark) {
  .m-hype { --m: #3987e5; } .m-generic { --m: #d95926; } .m-shouting
    { --m: #199e70; }
  .m-promo { --m: #c98500; } .m-incentive { --m: #d55181; }
}
```

File: train_model.py *(Model training script)*

```
"""Train the ML half of the detector and save it to
    models/review_model.joblib.

Dataset: "Fake Reviews Dataset" (Salminen et al., 2022,
    https://osf.io/tyue9/) -
40k Amazon reviews, half genuine (label OR) and half computer-
    generated (label CG).

Usage:
    python train_model.py                       # downloads the
    dataset if needed
    python train_model.py --data my_reviews.csv --text-col text --
    label-col label --fake-label 1
"""

from __future__ import annotations

import argparse
import urllib.request
from datetime import date
from pathlib import Path

import joblib
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
    f1_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import FeatureUnion, Pipeline

from detector.model import MODEL_PATH

DATASET_URL = "https://osf.io/download/3vds7/"
DEFAULT_DATA = Path("data/fake_reviews.csv")
```

```
def build_pipeline() -> Pipeline:
    features = FeatureUnion([
        ("words", TfidfVectorizer(ngram_range=(1, 2), min_df=3,
     max_features=60_000,
                                  sublinear_tf=True,
     strip_accents="unicode")),
        ("chars", TfidfVectorizer(analyzer="char_wb", ngram_range=(3,
     5), min_df=5,
                                  max_features=60_000,
     sublinear_tf=True)),
    ])
    return Pipeline([
        ("features", features),
        ("clf", LogisticRegression(C=4.0, max_iter=2000)),
    ])


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__,
     formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("--data", type=Path, default=DEFAULT_DATA)
    parser.add_argument("--text-col", default="text_")
    parser.add_argument("--label-col", default="label")
    parser.add_argument("--fake-label", default="CG")
    args = parser.parse_args()

    if not args.data.exists():
        if args.data != DEFAULT_DATA:
            raise SystemExit(f"Dataset not found: {args.data}")
        print(f"Downloading dataset to {args.data} ...")
        args.data.parent.mkdir(parents=True, exist_ok=True)
        urllib.request.urlretrieve(DATASET_URL, args.data)

    df = pd.read_csv(args.data).dropna(subset=[args.text_col,
     args.label_col])
    texts = df[args.text_col].astype(str)
    labels = (df[args.label_col].astype(str) ==
     str(args.fake_label)).astype(int)
    print(f"Loaded {len(df):,} reviews ({labels.sum():,} fake, {(1 -
     labels).sum():,} real)")

    x_train, x_test, y_train, y_test = train_test_split(
        texts, labels, test_size=0.2, stratify=labels,
     random_state=42)

    pipeline = build_pipeline().fit(x_train, y_train)
    proba = pipeline.predict_proba(x_test)[:, 1]
    pred = (proba >= 0.5).astype(int)
    metrics = {
        "accuracy": accuracy_score(y_test, pred),
        "f1": f1_score(y_test, pred),
        "roc_auc": roc_auc_score(y_test, proba),
        "test_size": len(y_test),
    }
    print(classification_report(y_test, pred, target_names=["real",
     "fake"], digits=3))
    print({k: round(v, 4) if isinstance(v, float) else v for k, v in
     metrics.items()})
```

```
    # Refit on everything for the shipped model; metrics above stay
     honest (held-out).
    pipeline = build_pipeline().fit(texts, labels)
    MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump({
        "pipeline": pipeline,
        "metrics": metrics,
        "dataset": "Fake Reviews Dataset (Salminen et al., 2022)",
        "n_reviews": len(df),
        "trained_on": date.today().isoformat(),
    }, MODEL_PATH, compress=3)
    print(f"Saved model to {MODEL_PATH} ({MODEL_PATH.stat().st_size /
     1e6:.1f} MB)")


if __name__ == "__main__":
    main()
```

File: tests/test_detector.py *(Unit tests)*

```
import dataclasses
from pathlib import Path

import pytest
from PIL import Image

from detector import analyze, ocr

SAMPLES = Path(__file__).resolve().parent.parent / "samples"


def test_hype_review_is_fake():
    result = analyze("AMAZING product!!! Best purchase ever, highly
     recommend to everyone. "
                     "Absolutely perfect, five stars!!! You won't
     regret it!", rating=5)
    assert result.is_fake


def test_spam_review_is_fake():
    assert analyze("Great product, works great. Use my code SAVE20 at
     www.dealz-shop.com!").is_fake


def test_spam_is_not_averaged_away_even_when_relaxed():
    # A promo code + shop link is damning on its own; the clean
     checks must not dilute it.
    result = analyze("Great product, works great. Use my code SAVE20
     at www.dealz-shop.com for a discount!")
    assert result.fake_probability >= 0.9
    assert dataclasses.replace(result, threshold=0.65).is_fake


def test_isbn_is_not_a_phone_number():
    result = analyze("Great book, ISBN 0307887898, the second half
     drags a little but worth reading.")
    assert not next(s for s in result.signals if s.key ==
     "promo").counts
```

```
@pytest.mark.parametrize("threshold", [0.35, 0.5, 0.65])
@pytest.mark.parametrize("text", [
    "Great product, works great. Use my code SAVE20 at www.dealz-
     shop.com!",
    "The lid is a bit stiff and the cable is short, but after two
     months it still boils fast.",
])
def test_confidence_never_below_half(text, threshold):
    assert dataclasses.replace(analyze(text),
     threshold=threshold).confidence >= 0.5


def test_detailed_balanced_review_is_real():
    result = analyze(
        "I've been using this kettle for about three months. It boils
     1.5 liters in roughly "
        "four minutes, which is faster than my old one. The lid is a
     bit stiff to open and the "
        "cable is short, but for the price it does the job well.")
    assert not result.is_fake


def test_rating_mismatch_fires():
    result = analyze("Broke after two days and support never replied.
     Terrible.", rating=5)
    assert next(s for s in result.signals if s.key ==
     "rating").triggered


def test_too_short_is_rejected():
    with pytest.raises(ValueError):
        analyze("Nice")


def test_ocr_reads_sample_screenshot():
    result = ocr.extract_review(Image.open(SAMPLES /
     "sample_fake_review.png"))
    assert "highly recommend" in result.text
    assert "helpful" not in result.text.lower()
    assert result.rating == 5
    assert result.verified_badge
```

File: tests/test_app.py *(End-to-end test)*

```
"""Smoke test: run the whole Streamlit page headless, the way
     Streamlit Cloud will."""

from pathlib import Path

from streamlit.testing.v1 import AppTest

APP = str(Path(__file__).resolve().parent.parent / "app.py")


def test_app_loads_and_analyzes_example():
    at = AppTest.from_file(APP, default_timeout=120).run()
```

```
assert not at.exception

at.button[0].click().run()          # "Suspicious" example fills
 the box and auto-analyzes
assert not at.exception
html = " ".join(m.value for m in at.markdown)
assert "Likely FAKE" in html
assert len(at.dataframe) == 1       # session history recorded
 the check
```

Appendix B: User Manual

B.1 Installing and running the application

1. Install Python 3.11 from python.org.
2. Download the project from GitHub, or run: git clone https://github.com/rahulpandey4950/fake-review-detector.git
3. Open a terminal in the project folder and create a virtual environment: python -m venv .venv
4. Activate it: .venv\Scripts\activate on Windows, or source .venv/bin/activate on Linux and macOS.
5. Install the libraries: pip install -r requirements.txt
6. Start the application: streamlit run app.py. The browser opens at http://localhost:8501.
7. Optional: run the tests with pip install pytest followed by pytest, or retrain the model with python train_model.py.

B.2 Checking a pasted review

1. Open the Paste review tab (Figure 4.1).
2. Paste the review into the Review text box, or click one of the Suspicious, Genuine or Spam examples.
3. Optionally choose the star rating that the reviewer gave.
4. Click Analyze review. The verdict card shows FAKE or REAL, the fake score and the confidence (Figure 4.2).
5. Use the Highlights, Criteria, Sentence mood and Details views to see why the verdict was given (Figures 4.2 to 4.4).

B.3 Checking a screenshot

1. Open the Upload screenshot tab and choose a PNG, JPG or WEBP image of a single review (Figure 4.5).
2. Check the extracted text and correct any OCR mistakes in the text box.
3. Check the detected star rating and change it if needed.
4. Click Analyze screenshot.

B.4 Choosing the strictness

Use the Strictness control in the sidebar. Relaxed flags only clear-cut cases (score 65% or more), Balanced is the default (50% or more) and Strict flags more reviews (35% or more). The verdict updates immediately without repeating the analysis.

B.5 Deploying on Streamlit Community Cloud

1. Push the project to a public GitHub repository, including the models folder.
2. Sign in at https://share.streamlit.io with the GitHub account and click Create app.
3. Select the repository and the main branch, and set the main file path to app.py.
4. Under Advanced settings, choose Python 3.11 and click Deploy. The application is published at a public streamlit.app address.

Appendix C: Plagiarism Report

[Attach the similarity report here.]

Appendix D: Weekly Progress Log and Team Contribution

Week	Dates	Work planned	Work completed	Guide's remarks	Guide's sign
1	[..]	[..]	[..]	[..]	
2	[..]	[..]	[..]	[..]	
3	[..]	[..]	[..]	[..]	
4	[..]	[..]	[..]	[..]	

Team contribution

Student	Modules coded	Report chapters written
[Name]	[..]	[..]
[Name]	[..]	[..]
[Name]	[..]	[..]
[Name]	[..]	[..]